

Essentials - read this first

Amazon ML Challenge 2026 Grand Finale - 7 October 2026

Team Androids - Presenter: Pardheev

The short companion of the full study guide (QA_STUDY_GUIDE.pdf): the pitch, the pipeline on one page, the 30 concepts a judge is most likely to probe, the 40 most likely questions with their spoken answers, the honest limitations, the key numbers and the to-do list. Final public F0.5: 0.991300 (say "0.9913").

How to use the Essentials

Read this booklet twice: once now, slowly and aloud, and once more on the morning of 7 October. It is the short version of the full study guide (QA_STUDY_GUIDE.pdf), and every answer, concept and number in it is copied word for word from that guide or from the talk's prep sheet, so the two never disagree.

The full guide is for depth: open it when you want the technical explanation of a concept, the numbers and sources behind an answer, or a question that is not in here.

- **Same numbers as the full guide.** A concept marked "(full: A1.4)" and a question marked Q16 have the same number there, with the complete version.
- **Questions.** *Why they ask* is what the judge is testing. **Say** is your spoken answer: direct answer first, then one number, then stop. Use **If they push** only if they ask again. ★ basic, ★★ probing, ★★★ hard; LIKELY = a question we expect.
- **Concepts.** Read *In one line* and *Remember* until you can say them without looking. Numbers marked toy or illustrative only teach the arithmetic: never quote them as results.
- **Before Tuesday evening,** settle the three items of the to-do list in [section 7](#).

Contents

1. The 30-second and 2-minute pitch	3
2. The pipeline on one page	4
Traps to avoid when you describe the pipeline	5
3. The 30 must-know concepts	6
The task and the metric	6
Finding candidates	7
The models	12
Deciding: thresholds and the decision rules	16
Training and validating without leaks	18
France, new data and model selection	21
4. The 40 most likely questions	24
5. Limitations and "if you don't know"	38
The honest-limitations answers	38
If you don't know: how to answer gracefully	39
Three answers with agreed wording	40
6. Key numbers	41
7. Before the finale: to do	42

1. The 30-second and 2-minute pitch

Taken word for word from the end of the pipeline walk-through (A4.20 of the full guide). Learn the 30-second version by heart; it is the answer to "so what did you build?". The 2-minute version is for a judge who asks you to walk through the whole pipeline.

One sentence (about 20 seconds). "Per country, a cheap index and two pruning steps cut 6.7 trillion possible pairs to about seven candidates per business, small language-model readers and a per-country decision model score them, and we link a record only to a confident, clear winner, with second-chance searches and French rules on top."

30 seconds (about 80 words). "Per country, we clean the text and a C++ inverted index pulls up businesses that share rare words, letter pieces or sounds, about 130 per business. A quick rule with no model cuts that to 21.7, and a CatBoost ranker keeps at most four per record. Cross-encoder readers score each pair, 'what changed' features describe the differences, and a LightGBM decision model links only a confident, clear winner. Second-chance searches, a consistency check and France rules finish it: public F0.5 0.9913."

2 minutes (about 300 words). "We link about ten million incoming records from the US, India and France to 1.7 million businesses, each record to one business or to none. The metric is F0.5 per business, and a business with no true records scores a point only if we link nothing, so we are precision first. The pipeline runs per country and goes from cheap to expensive.

First we clean the text: lower case, abbreviations unified, Indian-script words transliterated with a word list learned from training matches, and French address and legal-form rules. A C++ inverted index then finds businesses that share rare words, letter pieces or sounds, weighted by IDF, with separate lists for name and address so one-sided matches survive. That gives about 130 candidates per business. A quick rule-based filter with no model cuts that to 21.7, at a held-out cost of 0.00003 F0.5, and a CatBoost ranker keeps at most four per record: 6.75 per business.

Cross-encoder readers, fine-tuned multilingual MiniLM and E5, read each record and candidate together. We add 'what changed' features: which numbers changed, which words were added, dropped or replaced, how rare and how close in meaning, and whether the legal form changed. A LightGBM decision model per country turns about two hundred features into a probability, and we link only a confident, clear winner: at least 0.75, or 0.82 in France, and 0.4 ahead of the runner-up.

Then targeted fixes: two second-chance searches for unlinked records, by transliterated name in India and by rare address keys, adding 9,759 and 3,137 links; a per-business consistency check; and for France, which has no labels, self-trained French readers and rules computed by code.

The final file scores 0.9913 on the public leaderboard, and close versions of our decision models score between 0.990 and 0.992 on held-out folds."

If a judge then asks for limitations, add: "We had no untouched end-to-end validation of the final file, France has no labels and its rules were designed after looking at unlabelled test records, and we kept the French E5 reader although it failed two of its four written checks."

2. The pipeline on one page

Everything runs **per country** (US, India, France), and every search stays inside one country. A small configuration file per country, the **country profile**, picks the cleaning rules, readers, decision model, threshold, second-chance searches and rules; the code path is the same for every country (Doc 2.2).

```

incoming record (Source 2 or 3)                                businesses of the same country (Source 1)
[1] normalise text -----> [1] normalise text, build the index
[2] index lookup: name words, address words, letter pieces, sounds, exact core name
    |         about 130 candidates per business (blocking study)
[3] quick rule-based filter (no model) -> 21.71 per business
[4] CatBoost ranker -> shortlist (at most 4 per record) -> 6.75 per business
[5] readers (cross-encoders) score each shortlist pair
[6] "what changed" + other features (217 US / 211 India / 158 France)
[7] LightGBM decision model of that country -> a probability per pair
[8] confident + clear winner? yes -> link    no -> stay unlinked
US / India: [9] native-script second chance (India) -> [10] per-business consistency check -> [11] address second chance
France:     [12] France rules computed by code
[13] two output files, checked by the organisers' validator (7.10 candidates per business in the candidate file)
  
```

#	Stage	Runs for	What happens, in plain words	In, then out (real numbers)	Measured cost
1	Normalisation	all (France adds French rules)	Clean names and addresses so that the same thing is written the same way	raw text, then cleaned text for the index and features	not timed separately
2	Index lookup	all	Find businesses that share rare words, letter pieces or sounds	6,724,569,566,212 possible same-country pairs, then about 130.1 candidates per business (blocking study of 25 Sep)	stops 2-4 together on all 9.97 million test records: about 1.3 h
3	Quick rule-based filter	all	Drop clearly weaker candidates with fixed rules, no model	130.1, then 21.7107 per business (37,614,782 pairs)	no model; held-out F0.5 0.990117 to 0.990086
4	CatBoost ranker and shortlist	all	Order the candidates and keep at most four per record	21.71, then 6.75 per business (11,687,317 pairs; at most 4 per record, 1.17 on average) (6.75 and 1.17 derived)	inside the 1.3 h
5	Readers	all; which readers depends on the country	Small multilingual transformers read record and candidate together and give one score	one logit per reader per shortlist pair	about 1.4 h for two readers per pair (clean run)
6	Features	all	Describe exactly how the candidate differs: numbers, words, legal form	217 (US), 211 (India), 158 (France, default) features per pair	not timed separately
7	Decision model	one per country	One model per country turns about 200 features into a probability per candidate	a probability per pair	not timed separately
8	Confident + clear winner	all	Link the best candidate only if it is likely enough and clearly ahead	accepted: US 2,250,036; India 2,726,453; France 887,630	negligible
9	Native-script second chance	India	Search again for unlinked Indian records by transliterated name in the same state	171,713 records searched, +9,759 links	not timed separately
10	Consistency check	US, India	Re-decide with a bar that depends on how many links the business already has	US +277 / -284; India +247 / -248	negligible
11	Address second chance	US, India	Search again for unlinked records by rare numbers and rare address words	US +192; India +2,945 links	about 3.2 GB RAM per worker
12	France rules	France	No French labels: self-trained readers, a stricter bar and rules computed by code	887,630 accepted, then 867,559 links	not timed separately
13	Output and validator	all	Write the two files; check them with assertions and the organisers' validator	5,856,936 links; 12,293,019 candidate pairs (7.0954 per business); validator PASS	—

Sources: Doc 2.1, 3, 4, B.1, B.3; times from the clean-run report (CodeREADME 6, clean_run/RESULT.md).

The idea behind the order. Cheap steps at the start decide what gets read; expensive models at the end decide what gets linked. Each stage does the cheapest job it can do well, and we measured what each stage loses (QA 1, Script slide 4).

Traps to avoid when you describe the pipeline (full: A4.19)

1. "About 130 per business" comes from the **blocking study of 25 September**, not the final run. The final run reports 21.7107 after the quick filter.
2. 6.75 per business and 1.17 per record for the shortlist are **derived** (11,687,317 / 1,732,544 and / 9,969,589). The candidate file is 7.0954 per business, 1.2331 per record.
3. The CatBoost ranker is a **pointwise classifier** with log loss. Never say it uses a ranking loss.
4. The quick filter's cost (0.990117 to 0.990086) was measured **with the decision model of 25 September**.
5. **France has no consistency check and no second-chance searches**, does not use the plain E5 reader, and its decision model has **no French rows**.
6. **The second-chance searches never undo a link; the consistency check can** (US -284, India -248).
7. The real order for the US and India is: native-script second chance (India), then the consistency check, then the address second chance. The talk groups the two searches together for simplicity.
8. The readers read the **raw** text; the index and hand-made features use the cleaned text.
9. "1.4 hours of reader scoring" is from the **clean run** (two readers per pair, all except the French E5 reader); the submitted run reused stored logits.
10. Held-out scores (US 0.991212 / 0.991545; India 0.990046 / 0.990298) are for **close versions** of the decision models (the India recipe with 3 of its 5 seeds) and **exclude** the second-chance searches and France rules.
11. The 0.4 lead is **one fixed value for every country**; the threshold 0.75 was tuned on fold 8. Do not claim the lead was tuned, and do not say it was never tuned.
12. The unseen-country route is **untested** on real data.
13. There is **no measured French F0.5**; the France "proxy" numbers are US and Indian records.
14. Never use internal code names on stage. For hard near-identical cases use the QA 15 wording ([Q115](#)), and only if asked.
15. The cap of four rarely binds (1.17 pairs per record on average); do not say the cap is what cuts the candidates down. The quick filter and the 0.001 floor do most of the narrowing; the cap bounds the worst case.
16. The break-even arithmetic of the consistency check explains only the **direction** of its shape; the four bars were **tuned on folds 8 and 9**. Check the Stop 10 table for the metric's own numbers.
17. Agreement with held-out French pseudo-labels (0.999206) shows that the student matches its teacher; it is **not** evidence of French accuracy.
18. The index stage on test runs with 6 threads by default; the "8-11 threads" in the code README refer to the training-stage index runs.

3. The 30 must-know concepts

The concepts a judge is most likely to probe, in six groups. Each has its *In one line*, the *Simple explanation*, a short *Worked example* where the guide has one, and the *Remember* points, copied word for word from the full guide; the number after "full:" is where the complete concept is (with the technical explanation, how we used it and the expert nuance).

The task and the metric

1. F-beta and why F0.5 (full: A1.4)

IN ONE LINE

F-beta combines precision and recall into one number, and $\beta = 0.5$ makes precision count more than recall.

SIMPLE EXPLANATION

You cannot optimise two numbers at once, so you need one. A plain average would flatter silly systems: precision 1.0 with recall 0.01 averages to 0.505. F-beta uses a **harmonic mean** (two divided by the sum of the two reciprocals), which is dragged down by whichever of the two is worse: for the same system it is $2 \times 1.0 \times 0.01 / (1.0 + 0.01) = 0.0198$. Beta is a dial: $\beta = 1$ treats precision and recall equally (F1), $\beta = 2$ favours recall, $\beta = 0.5$ favours precision. The competition chose 0.5, which says: a wrong link hurts more than a missed one.

WORKED EXAMPLE

(a) Two systems with the same F1:

System	Precision	Recall	F1	F0.5	F2
Careful	0.9	0.6	0.720	0.818	0.643
Greedy	0.6	0.9	0.720	0.643	0.818

(b) Count form, $TP = 2$, $FP = 0$, $FN = 1$: $F0.5 = 1.25 \times 2 / (1.25 \times 2 + 0.25 \times 1 + 0) = 2.5 / 2.75 = 0.909$, while $F1 = 2 \times 2 / (2 \times 2 + 1 + 0) = 0.800$.

(c) A 1 % loss on each side: precision 1.00 with recall 0.99 gives $F0.5 = 1.25 \times 0.99 / (0.25 + 0.99) = 0.998$; precision 0.99 with recall 1.00 gives $1.25 \times 0.99 / (0.2475 + 1) = 0.992$. Losing 1 % of precision costs about four times as much as losing 1 % of recall.

REMEMBER

- $F0.5 = 1.25 \cdot P \cdot R / (0.25 \cdot P + R) = 1.25 \cdot TP / (\text{predicted} + 0.25 \cdot \text{true})$.
- In the denominator one wrong link weighs as much as four missed links.
- A link helps only if its chance of being right exceeds about $F / 1.25$ (pooled, calibrated sanity check).

2. Macro-averaged per-business F0.5 and the empty-row rule (full: A1.5)

IN ONE LINE

The competition computes F0.5 separately for every business and then takes the plain average, and a business with no true records scores 1 only if we link nothing to it.

SIMPLE EXPLANATION

A teacher can grade a class in two ways: pool all marks into one big total (**micro-averaging**), or grade each student separately and average the grades (**macro-averaging**). Our competition grades each business like a student and averages, so a tiny business with one record counts exactly as much as a big one. A business that truly owns no records is like a student with no homework due: full marks only if you hand them nothing, zero if you hand them one wrong paper. That is why one wrong link can turn a perfect 1 into a 0.

WORKED EXAMPLE

Per-business formula: $F_b = 1.25 \cdot TP_b / (pred_b + 0.25 \cdot true_b)$, and $F_b = 1$ when $pred_b = true_b = 0$.

Business	True records	Linked	Correct	F_b
A	2	2	2	$1.25 \times 2 / (2 + 0.5) = 1.000$
B	0	0	0	1 (empty-row rule)
C	0	1	0	$0 / (1 + 0) = 0.000$

1. Macro average = $(1 + 1 + 0) / 3 = 0.667$.
2. Micro (pooled: TP 2, predicted 3, true 2) = $1.25 \times 2 / (3 + 0.5) = 0.714$.
3. The single wrong link costs a full third of the macro score here.
4. A business D with 3 true records scores 0.909 with 2 correct links, 1.000 with 3 correct links, and 0.667 with 2 correct links plus 1 wrong one.

REMEMBER

- $F_b = 1.25 \cdot TP_b / (pred_b + 0.25 \cdot true_b)$; empty truth plus empty prediction = 1.
- Every business weighs the same: one wrong link into an empty business costs a whole business-point.
- Tune and compare on the exact metric, per business, with paired resampling over businesses.

Finding candidates

3. Transliteration of Indian native scripts (full: A1.10)

IN ONE LINE

Transliteration rewrites a word from one script into another (विजय → vijay); we learned the mapping for business-name words from labelled training matches instead of relying on a generic romaniser.

SIMPLE EXPLANATION

If someone spells a name to you over a noisy phone line, you write it in English letters as best you can, and different people produce different spellings. Generic romanisers work letter by letter and often produce strange spellings, especially for English words written in Hindi letters: "एंटरप्राइजेज" is simply the English word "enterprises". Our training data already contained many correct pairs: a Devanagari incoming record and its business written in English letters. We counted which Latin word appears again and again with each Hindi word and built a small dictionary (a **lexicon**) from those counts, like learning a language from bilingual shop signs. Transliteration changes the script, not the language; **translation** would change the language.

WORKED EXAMPLE 1 (COMPUTED; THE LEXICON OUTPUT IS DOC D.2)

Native word	Generic romaniser (unicode, lower-cased)	Learned lexicon
विजय	vijy	vijay
एंटरप्राइजेज	enttrpraaijej	enterprises
प्राइवेट	praaivett	private

Native word	Generic romaniser (unicode, lower-cased)	Learned lexicon
लिमिटेड	limitedd	limited

The generic output shares no word with "Vijay Enterprises Private Limited"; the lexicon output is identical to it.

WORKED EXAMPLE 2 (ILLUSTRATIVE COUNTS): HOW ONE LEXICON ENTRY IS LEARNED

Suppose the token विजय appears in 40 labelled training records. In the true business names of those records, the Latin word "vijay" appears 38 times, "enterprises" 20 times and "traders" 6 times. Across all native-script training examples, these Latin words appear in business names 60, 2,000 and 900 times. Score = co-occurrences / (Latin word count + 1)^{0.6}:

Latin word	Co-occurrences	(count + 1) ^{0.6}	Score
vijay	38	61 ^{0.6} = 11.78	3.225
enterprises	20	2,001 ^{0.6} = 95.66	0.209
traders	6	901 ^{0.6} = 59.27	0.101

Checks: co-occurrences at least 3 (38, yes); purity 38 / 40 = 0.95, at least 0.35 (yes); best score at least 1.8 times the runner-up (3.225 against 1.8 × 0.209 = 0.376, yes). So the lexicon gets the entry विजय → vijay. Dividing by the Latin word's frequency stops very common words such as "enterprises" from winning for every Hindi word.

REMEMBER

- Generic romanisation: विजय → "vijy", एंटरप्राइजेज → "enttrpraiiej"; the learned lexicon gives "vijay enterprises".
- Lexicon = co-occurrence counts on labelled folds 0-5, kept if $n \geq 3$, purity ≥ 0.35 and score $\geq 1.8 \times$ runner-up; unicode otherwise.
- The rescue lexicon leaves each record's own fold out; the label-free test alignment (probability ≥ 0.9) is disclosed.

4. Blocking and candidate generation: reduction ratio, pair completeness, pairs quality (full: A1.19)

IN ONE LINE

Blocking (candidate generation) quickly picks, for each incoming record, a small set of plausible businesses, so that the expensive comparison runs only on those pairs.

SIMPLE EXPLANATION

A librarian asked for "Ganesh Traders, Pune" does not read all 1.7 million catalogue cards; she opens the catalogue at "Ganesh" and at "Pune" and pulls out a handful. Blocking is that catalogue step. It must be fast, it must be small (few cards pulled), and it must be safe (the right card is almost always among them). If the right card is not pulled, no amount of careful reading later can find it. The name comes from the oldest form of the idea: put records into "blocks" that share a **blocking key** (for example the same postcode) and compare only inside a block.

The words you need: the **comparison space** is the set of all possible pairs; the **reduction ratio (RR)** is the share of that space we do not keep; **pair completeness (PC)** is the share of true matches we keep (the recall of blocking); **pairs quality (PQ)** is the share of kept pairs that are true matches (the precision of blocking).

WORKED EXAMPLE

Toy (illustrative): 1,000 records $\times$ 1,000 businesses = 1,000,000 pairs, with 1,000 true matches. Blocking keeps 3,000 pairs, 950 of them true matches.

1. $RR = 1 - 3,000 / 1,000,000 = 0.997$.
2. $PC = 950 / 1,000 = 0.95$.
3. $PQ = 950 / 3,000 = 0.317$.

Our test set (Doc 3; per-country products derived):

Country	Businesses $\times$ incoming records	Pairs
US	663,106 $\times$ 3,817,031	2,531,096,158,286
India	809,986 $\times$ 4,717,565	3,821,161,604,090
France	259,452 $\times$ 1,434,993	372,311,803,836
All		6,724,569,566,212

The candidate file holds 12,293,019 pairs, so $RR = 1 - 12,293,019 / 6,724,569,566,212 = 0.99999817$: about 1.83 pairs kept per million (derived). PC at the index lookup on labelled fold 9: 99.3694 % (US) and 98.5068 % (India).

REMEMBER

- Blocking = a cheap search that bounds what the expensive models see; it sets the recall ceiling.
- RR 0.99999817 (12,293,019 of 6,724,569,566,212 pairs); read it only together with recall.
- Pair completeness at the index lookup on fold 9: 99.37 % US, 98.51 % India.

5. The inverted index (full: A1.20)

IN ONE LINE

An **inverted index** maps every word (or letter piece) to the list of businesses that contain it, so candidates are found by looking words up instead of scanning everything.

SIMPLE EXPLANATION

It is the index at the back of a book: "Ganesh: pages 12, 57". To find businesses that share words with an incoming record, look up each of the record's words, collect the businesses listed, and give each business points for every word it shares, more points for rarer words. Very common words, like "the" in a book, have enormous lists and say little, so we skip lists that are too long and stop once we have read enough.

The words you need: a **posting list** is the list of businesses for one word; one entry of it is a **posting**; an **accumulator** is the running score of each business during a lookup; a **posting-list cap** is the longest list we are willing to read; a **budget** is the total number of postings we read per record and channel.

WORKED EXAMPLE

A tiny index of three businesses: B1 "shree ganesh traders", B2 "ganesh sweets", B3 "vijay traders".

Word	Posting list
ganesh	B1, B2
shree	B1
sweets	B2
traders	B1, B3
vijay	B3

Query "shri ganesh traders":

1. "shri" has no list (no business contains it).
2. "ganesh" adds IDF(ganesh) to B1 and B2.
3. "traders" adds IDF(traders) to B1 and B3.
4. Accumulators: B1 has both words, B2 and B3 one each, so B1 ranks first. We read 4 postings instead of comparing against every business.

Caps and budget (list lengths illustrative; the limits are the real name-channel values, cap 14,000 per list and budget 22,000 per record), with words processed rarest first:

Query word	List length	Action	Postings used so far
vijay	3,000	read	3,000
shree	9,000	read	12,000
traders	12,000	skipped: 12,000 + 12,000 would exceed the 22,000 budget	12,000
enterprises	40,000	skipped: longer than the 14,000 cap	12,000

REMEMBER

- Posting list = term → businesses containing it; a lookup adds IDF for every shared term.
- Rarest first; skip lists longer than the cap; stop at the budget: bounded work per record.
- Deterministic and auditable (a fixed pruning rule); C++ with OpenMP; about 1.3 hours for lookup, filter and ranker on test.

6. Tokens, sets and character n-grams (full: A1.12)

IN ONE LINE

To compare text we cut it into pieces, whole words (**tokens**) or short overlapping letter chunks (**character n-grams**), and compare the collections of pieces.

SIMPLE EXPLANATION

Comparing two names letter by letter is fragile, because one typo pushes every later letter out of line. Instead, cut each name into pieces and count the pieces they share, like comparing two Lego models by the bricks they use. Words are big pieces: "shree ganesh" and "shri ganesh" share only one of their three different words. Letter chunks are small pieces ("shr", "hre", "ree", ...): a typo spoils only a few of them, so the two names still share most chunks. A **set** keeps each piece once and ignores order; a **bag** (multiset) also counts repeats. An n-gram of 3 letters is a 3-gram (also called a q-gram or shingle).

WORKED EXAMPLE (COMPUTED WITH OUR INDEX RULES; LETTER PIECES ARE CUT FROM THE CORE NAME WITH SPACES REMOVED, SOUND PIECES FROM ITS WORDS SORTED ALPHABETICALLY)

"shree ganesh" against "shri ganesh".

Pieces	shree ganesh	shri ganesh	Shared	Union	Jaccard
Words	shree, ganesh	shri, ganesh	1	3	0.333
3-grams	shr hre ree eeg ega gan ane nes esh (9)	shr hri rig iga gan ane nes esh (8)	5	12	0.417
4-grams	shre hree reeg eega egan gane anes nesh (8)	shri hrig riga igan gane anes nesh (7)	3	12	0.250
Sound 4-grams (see Phonetic keys)	gnsh nshs shsh hshr	gnsh nshs shsh hshr	4	4	1.000

A second case: "book world" against "bookworld" share no word, but their compacted letter pieces are identical.

REMEMBER

- Words are precise but brittle; letter n-grams survive typos and split or merged words.
- One edit spoils at most n of a string's n-grams (the q-gram lemma).
- Our index uses words, 3-grams, 4-grams and sound 4-grams, each with its own weight and caps.

7. IDF, TF-IDF, BM25 and our IDF-weighted overlap (full: A1.16)

IN ONE LINE

IDF (inverse document frequency) gives rare words more weight than common ones, because sharing a rare word is much stronger evidence of a match.

SIMPLE EXPLANATION

In a phone book, two entries sharing the word "Traders" tells you almost nothing, because thousands of shops are traders; two entries sharing "Legacy" tells you a lot. The **document frequency (df)** of a word is the number of records that contain it; IDF turns "few records contain it" into "high weight". **TF** (term frequency) counts how often a word appears inside one record. **TF-IDF** and **BM25** are standard search-engine formulas that combine both; ours is a simpler cousin built for short names, which uses IDF only.

REMEMBER

- $IDF = \log(1 + (N + 1) / (df + 1))$: rare words weigh more; the minimum is $\ln 2$.
- Our retrieval score is an IDF-weighted set overlap with per-channel length penalties: no TF, not BM25, not cosine.
- IDF statistics come from the unlabelled records at run time (label-free, disclosed).

8. Recall ceiling and the candidate funnel (full: A1.23)

IN ONE LINE

The **recall ceiling** is the share of true matches still present among the candidates after each step; no later model can recover what candidate generation lost.

SIMPLE EXPLANATION

Imagine panning for gold with a stack of sieves: each layer removes sand, and after every layer we count how many gold grains are left. If gold is lost at layer two, no clever work at layer five can bring it back, except a separate second search. So we counted the gold after every layer: that is the candidate **funnel** with its recall at each stage.

WORKED EXAMPLE (DOC B.2, TRAIN FOLD 9; MISSED COUNTS DERIVED)

Stage	US recall	India recall	Owned records missed (derived)
Index lookup	99.3694 %	98.5068 %	about 2,888 of 457,971 (US); about 4,536 of 303,786 (India)
Shortlist (quick filter, then top 4), lower to upper bound	98.8748 % to 99.3628 %	98.0572 % to 98.4831 %	about 2,918 to 5,153 (US); about 4,608 to 5,902 (India)

Candidates per incoming record in the all-fold samples: index 27.541 (US) / 23.145 (India) → after the filter 3.111 / 3.878 → shortlist 1.114 / 1.174.

Why recall losses hurt less than precision losses: with perfect precision, a recall of 0.99 still gives $F0.5 = 1.25 \times 0.99 / (0.25 + 0.99) = 0.998$ (pooled; see F-beta).

REMEMBER

- Index lookup 99.37 % US / 98.51 % India; shortlist at least 98.87 % / 98.06 % (fold 9).
- The cap of four is the largest loss; the quick filter costs very little.

- Second-chance searches exist because a missing owner cannot be fixed by any threshold.

The models

9. Gradient boosting, step by step (full: A2.5)

IN ONE LINE

Gradient boosting builds a team of small trees one after another, where each new tree is trained to correct the mistakes the team still makes.

SIMPLE EXPLANATION

Think of putting in golf. The first putt gets the ball near the hole; each following putt only has to cover the remaining distance; and you deliberately putt a little short each time so you never overshoot wildly. In gradient boosting the first "putt" is a constant guess, each new tree is fitted to the remaining error (the **residual**), and its contribution is shrunk by the **learning rate** so that many small corrections add up. To score a pair, you add the outputs of all trees and pass the total through the sigmoid to get a probability.

REMEMBER

- Each new tree corrects the current errors: $\text{target} = \text{label} - \text{predicted probability}$.
- Leaf value = $-(\text{sum of gradients}) / (\text{sum of Hessians} + L2)$; learning rate shrinks every step.
- Final score = sum of all trees, passed through the sigmoid.

10. CatBoost: symmetric trees, ordered boosting, categorical encoding (full: A2.7)

IN ONE LINE

CatBoost is a gradient-boosting library whose trees ask the same question at every level (symmetric trees) and which offers special machinery, ordered boosting and ordered target statistics, against a subtle kind of training leakage.

SIMPLE EXPLANATION

A **symmetric tree** (also called an **oblivious tree**) is like a fixed questionnaire: every pair answers the same 7 questions in the same order, and the 7 yes/no answers form a 7-digit binary code that points to one of 128 boxes, each holding a score. It is quick to apply and hard to overfit. **Ordered boosting** is like marking each student's homework with an answer key built only from students who handed in before them, so nobody's mark is influenced by their own answers. **Ordered target statistics** turn a text category such as "city = Nantes" into a number using only earlier rows, for the same reason.

WORKED EXAMPLE (TOY NUMBERS)

- *A symmetric tree of depth 2.* Level 1 asks "name similarity ≥ 0.8 ?" (bit b1). Level 2 asks "address coverage ≥ 0.95 ?" (bit b2) on both branches. The leaf index is $2 \times b1 + b2$, and the leaf table is $[-2.1, -0.4, 0.3, 1.9]$. A pair with similarity 0.9 ($b1 = 1$) and coverage 0.5 ($b2 = 0$) lands in leaf 2 and gets 0.3. Our ranker's trees work the same way with 7 questions and 128 leaves: 7 comparisons and one table look-up per tree.
- *An ordered target statistic.* Rows arrive in a random order with a category and a label: Nantes (1), Pessac (0), Nantes (0), Nantes (1). With a prior of 0.5 and a prior weight of 1, each row is encoded as $(\text{sum of earlier labels of its category} + 0.5) / (\text{count of earlier rows of its category} + 1)$: row 1 Nantes = $0.5/1 = 0.5$; row 2 Pessac = 0.5; row 3 Nantes = $(1 + 0.5)/(1 + 1) = 0.75$; row 4 Nantes = $(1 + 0 + 0.5)/(2 + 1) = 0.5$. No row ever sees its own label.

REMEMBER

- Symmetric tree: same question per level; depth 7 means 7 comparisons and one look-up among 128 leaves.
- Our ranker: plain boosting, numeric features only, GPU, depth 7, up to 1,600 trees, learning rate 0.075.
- GPU training is fast but not bit-deterministic.

11. Learning to rank: pointwise, pairwise, listwise (full: A2.9)

IN ONE LINE

Learning to rank trains a model to order a list of candidates so the right one is near the top; it can learn from single items (pointwise), from pairs of items (pairwise) or from whole lists (listwise).

SIMPLE EXPLANATION

Picture a librarian who, for each question, must pull books off the shelf in order of usefulness. She could grade each book on its own ("useful or not": **pointwise**), compare books two at a time ("is this one better than that one?": **pairwise**), or judge the order of the whole pile at once (**listwise**). In our pipeline the ranker is like a shortlisting committee whose main job is not to leave the best applicant off a short interview list: what matters most is that the true business is among the few candidates the expensive readers will see.

REMEMBER

- Pointwise = score items alone; pairwise = compare two; listwise = optimise the whole list (e.g. NDCG).
- Our ranker is pointwise (Logloss); its job is recall at the shortlist (top 1 always, ranks 2-4 if at least 0.001).
- The cap of 4 is the largest recall loss: US 99.37 % at the index versus at least 98.87 % in the shortlist.

12. LightGBM: histograms and leaf-wise growth (full: A2.8)

IN ONE LINE

LightGBM is a fast gradient-boosting library that sorts feature values into a small number of bins (histograms) and grows each tree leaf by leaf, always splitting the leaf where the gain is largest.

SIMPLE EXPLANATION

Instead of testing every exact value of a feature as a possible question, LightGBM first groups the values into up to 255 buckets, like grouping exam marks into ranges, and only tests the bucket boundaries; that is much faster and loses almost nothing. Then, instead of growing a tree layer by layer like a pyramid, it behaves like a gardener who always works on the one branch that will improve the plant most: **leaf-wise growth**. The result is deep, uneven trees that capture the most useful combinations of features first.

WORKED EXAMPLE (TOY NUMBERS)

- *Histograms*. A feature such as "address coverage" over 1,000,000 training rows is binned into 255 bins. To find the best split, LightGBM adds up the gradients and Hessians of the rows in each bin once (one pass over the rows), then scans 254 bin boundaries instead of up to a million distinct values. After splitting a node, it builds the histogram of the smaller child only and gets the larger child's by subtraction: parent histogram minus smaller child = larger child.
- *Leaf-wise versus level-wise*. A tree currently has three leaves whose best possible splits would gain 5.0, 0.4 and 2.1. Level-wise growth would split all three. Leaf-wise growth splits only the leaf with gain 5.0, recomputes, and continues until the tree has 127 leaves (our setting). The budget of leaves is spent where it helps most.

REMEMBER

- Histograms: bin each feature (default 255 bins), search only bin boundaries; subtraction trick.
- Leaf-wise growth: always split the best leaf; capped at 127 leaves, at least 200 rows per leaf.
- Our decision models: learning rate 0.05, feature fraction 0.9, bagging 0.8, L2 1.0, mean of 5 seeds.

13. Bi-encoders versus cross-encoders (full: A2.13)

IN ONE LINE

A bi-encoder turns each text into a vector separately and compares the vectors; a cross-encoder reads both texts together and outputs one match score, which is more accurate but must be run once for every pair.

SIMPLE EXPLANATION

A **bi-encoder** is like comparing passports by measurements written on index cards: you measure every person once, and comparing two cards is instant, but anything not on the card is lost. A **cross-encoder** is like an examiner who holds both passports side by side and studies them together: slow, because she must sit with every pair, but she notices that exactly one letter of the name or the legal form differs. Our pipeline uses cheap steps to pick very few pairs, so the careful examiner only sees about one pair per incoming record.

REMEMBER

- Bi-encoder: encode separately, compare vectors, precomputable, good for retrieval.
- Cross-encoder: read jointly, one score per pair, more accurate, cost proportional to pairs.
- Our funnel makes cross-encoders affordable: 11.7 million shortlist pairs, about one per record.

14. MiniLM and knowledge distillation (full: A2.15)

IN ONE LINE

MiniLM is a small transformer trained to imitate a bigger one (knowledge distillation), and the multilingual MiniLM we used was further distilled so that a sentence in any of 50-plus languages lands near the vector of its English meaning.

SIMPLE EXPLANATION

Knowledge distillation is like an apprentice learning from a master craftsman: instead of learning only from right-or-wrong answers, the apprentice copies how the master works, including the master's uncertainty ("70 % this, 20 % that"). MiniLM's apprentice copies something specific: where the master's attention goes. The multilingual version then adds a translation lesson: a bilingual student learns to produce, for a French or Hindi sentence, the same summary vector that an English teacher produces for the English version. The result is a small, fast model that reads many languages.

WORKED EXAMPLE (TOY NUMBERS, PLUS REAL PARAMETER COUNTS)

- *Copying attention.* For one token, the teacher's attention over three tokens is (0.7, 0.2, 0.1) and the student's is (0.5, 0.3, 0.2). The distillation loss is the **KL divergence**, $\sum \text{teacher} \times \ln(\text{teacher} / \text{student}) = 0.7 \ln(1.4) + 0.2 \ln(0.667) + 0.1 \ln(0.5) = 0.236 - 0.081 - 0.069 = 0.085$. Training pushes it towards 0.
- *Copying sentence vectors across languages.* The English teacher gives (0.6, 0.8, 0.0) for an English sentence; the student gives (0.5, 0.7, 0.2) for its French translation. The loss is the mean squared error: $(0.1^2 + 0.1^2 + 0.2^2) / 3 = 0.02$.
- *Where MiniLM's 117,653,760 parameters sit (background, derived from the shipped configuration):* 96,212,352 in the embedding tables (82 %), 21,293,568 in the 12 transformer layers, and 147,840 in a

pooler layer our reader does not use.

REMEMBER

- Distillation: a small student imitates a big teacher (MiniLM copies attention patterns).
- Our MiniLM: 12 layers, hidden size 384, 117.7 million parameters (82 % embeddings), Apache-2.0, 50-plus languages.
- Used as the MiniLM reader (all countries), French MiniLM reader, shared tokenizer, and frozen word encoder.

15. E5 and contrastive pre-training (full: A2.16)

IN ONE LINE

E5 is a text-embedding model trained contrastively: it learns to place matching text pairs close together and non-matching pairs far apart.

SIMPLE EXPLANATION

Imagine a room full of question cards and answer cards. Each question card must find its own answer card among all the others; the model is rewarded when the true partner scores highest and punished when a stranger scores higher. With huge rooms (very large batches), every other card in the room is a free example of a wrong partner. This is **contrastive learning**. After training on an enormous number of such pairs, texts with the same meaning end up close together in the vector space.

WORKED EXAMPLE (TOY NUMBERS)

One query card and three answer cards in the batch, with cosine similarities 0.9 (its true partner), 0.5 and 0.3. The **temperature** tau sharpens the comparison; we use $\tau = 0.1$ here for readable numbers (E5 itself uses 0.01, background).

1. Divide by tau: 9, 5, 3.
2. Softmax: $e^9 = 8,103$, $e^5 = 148$, $e^3 = 20$; the true partner's share is $8,103 / 8,272 = 0.980$.
3. Loss = $-\ln(0.980) = 0.021$: small, because the true partner already wins.
4. If instead the true partner had 0.5 and a stranger 0.9, the true share would be 0.018 and the loss $-\ln(0.018) = 4.02$: large, so training pulls the true pair together and pushes the stranger away.

REMEMBER

- Contrastive learning: pull true pairs together, push in-batch strangers apart (InfoNCE with a temperature).
- multilingual-e5-base: XLM-R base backbone, 278.0 million parameters, MIT.
- E5 reader for the US and India; French E5 reader for France (weight 0.5 in the blend); no prefixes used.

16. Stacking: a two-level model with separate folds per layer (full: A2.23)

IN ONE LINE

Stacking puts a second model on top of first-level models: the CatBoost ranker's probability and the readers' logits become inputs of the LightGBM decision model, which is trained on different businesses so it learns how far to trust them.

SIMPLE EXPLANATION

A hiring panel's chair (the **meta-model**, our decision model) listens to specialists (the **base models**: the ranker and the readers) and also reads the plain facts of the CV (our features). To learn whose opinion to trust, the chair must watch the specialists judge candidates they have never met; if the chair only watched them re-judge candidates they had already studied, every specialist would look perfect and the chair

would learn to over-trust them. So the specialists and the chair are trained on different groups of businesses.

REMEMBER

- Base models (ranker, readers) on folds 0-5; meta-model (decision model) on folds 6-9.
- Base scores on the meta-model's rows must be out-of-sample, or the meta-model over-trusts them.
- The decision model adds what readers cannot see: competition between candidates and "what changed".

17. Calibration: what a model's "probability" means (full: A2.26)

IN ONE LINE

A model is calibrated if, among all candidates it scores about 0.8, about 80 % are true matches; our decision rule does not assume calibration, because its thresholds are tuned directly on the competition metric.

SIMPLE EXPLANATION

A weather forecaster is **calibrated** if it rains on about 70 % of the days she says "70 %". A forecaster trained in another city, or on a deliberately unusual set of days, may be systematically off. Our decision models were trained on specially chosen rows, and France's model on other countries, so instead of trusting "0.75 means 75 %", we choose the cut-off that scores best on held-out businesses with the real metric. Because the cut-off is chosen this way, any consistent over- or under-confidence is absorbed.

WORKED EXAMPLE (TOY NUMBERS)

1. *Reliability check.* Of 1,000 candidates scored between 0.80 and 0.90, if 850 are owners the model is calibrated there; if only 700 are, it is over-confident.
2. *A threshold does not care about the scale.* Linking when $p \geq 0.75$ is the same decision as linking when $\text{logit} \geq \ln(0.75 / 0.25) = 1.099$, and the same as any other monotone rescaling of the score with the threshold moved accordingly.
3. *The margin does care.* Best candidate 0.95, runner-up 0.60: lead 0.35, below 0.4, so no link. After the monotone rescaling $p \rightarrow p^2$ the scores become 0.9025 and 0.36, a lead of 0.5425: link. Same ranking, different decision. So whenever a model changes, the threshold and the margin must be re-checked on held-out data.

REMEMBER

- Calibrated = scores match observed frequencies; ours are not separately calibrated.
- Thresholds tuned on the exact per-business F0.5 absorb miscalibration; the 0.4 margin does not, so both are validated.
- France uses the conservative 0.82 because its calibration cannot be measured.

Deciding: thresholds and the decision rules

18. Tuning thresholds on held-out folds (full: A3.12)

IN ONE LINE

Choose the score cut-off for linking by trying several values on a held-out fold and keeping the one that gives the best exact competition score there, not the best AUC.

SIMPLE EXPLANATION

The decision model gives every candidate a probability, but you still need a rule: "link if the score is above X". If X is too low you make many wrong links (precision falls); if X is too high you miss true links (recall falls). The best X depends on how the scoring works. Our competition scores each business separately with F0.5, whose formula counts a wrong link four times as heavily as a missed one, then averages over businesses, and gives a full point to a business with no true records only if we link nothing to it. So we try candidate values of X on fold 8, compute exactly that score for each, and keep the best. A general-purpose measure such as AUC cannot do this job, because AUC does not depend on X at all.

REMEMBER

- Per-business F0.5 = $1.25 \times \text{TP} / (\text{predicted} + 0.25 \times \text{true})$; 1 for a correctly empty business.
- Thresholds were tuned on the exact metric on held-out folds: 0.75 on fold 8; France 0.82 without labels.
- AUC is threshold-free and pair-level; it cannot choose an operating point.

19. The confident + clear-winner rule (full: A3.15)

IN ONE LINE

Link a record to its best candidate only if that candidate's score is high enough (at least 0.75 in the US and India, 0.82 in France and for unseen countries) and at least 0.4 ahead of the runner-up; otherwise link nothing.

SIMPLE EXPLANATION

A careful detective names a suspect only when the evidence is strong and nobody else is nearly as likely. If two suspects look almost equally guilty, naming either one risks accusing the wrong person, and saying "not enough evidence" is the safer call. Under F0.5 a wrong link hurts more than a missed one, and a business with no true records keeps its full point only if nothing is linked to it. So "abstain when unsure" is the right default. The rule has two parts: **confident** (the score is above a threshold) and **clear winner** (the lead over the second-best candidate is at least 0.4). This kind of "abstain unless sure" rule is called a **reject option**.

WORKED EXAMPLE

(ours) The US record "Legacy Marketing Enterprises (Inc)", with no address, has four shortlisted businesses:

Business	Decision-model score
Legacy Marketing Enterprises Inc, Windsor, CT	0.965022
Legacy Marketing Enterprises PC, Joshua Tree, CA	0.088664
Legacy Marketing Industries Inc, Louisville, TN	0.000088
Legacy Marketing Enterprises LLC, Irondequoit, NY	0.000007

1. Best = 0.965022 $\geq$ 0.75: confident.
2. Lead = 0.965022 – 0.088664 = 0.876 $\geq$ 0.4: clear winner.
3. Link to the Windsor business, and to nothing else.

(toy) Three other records:

Record	Best	Second	Confident?	Clear winner?	Decision
X	0.81	0.62	yes	no (lead 0.19)	no link
Y	0.78	none (only one candidate)	yes	yes (runner-up treated as minus infinity)	link
Z	0.70	0.05	no	yes	no link

(toy) Why abstaining is cheaper: a business owns 3 records and we linked 2 correctly: $F0.5 = 2.5 / (2 + 0.75) = 0.909$. Add one wrong link: $2.5 / (3 + 0.75) = 0.667$. A loss of 0.24 from a single wrong link; on an empty business, one wrong link turns 1 into 0.

REMEMBER

- Link only if score ≥ 0.75 (0.82 France) and lead ≥ 0.4 ; at most one link per record.
- The margin makes up for pointwise scores that are not exclusive across candidates.
- Abstaining is cheap under F0.5; one wrong link can turn a correct empty row from 1 into 0.

20. Count-dependent thresholds: the per-business consistency check (full: A3.16)

IN ONE LINE

After the main decision, each record is re-decided with a bar that depends on how many other records were already linked to the same business: 0.75 if none, 0.68 if one, 0.72 if two, 0.77 if three or more, always with the 0.4 lead.

SIMPLE EXPLANATION

The value of one more link depends on what the business already has. If a business has no links yet, it might be one of the businesses with no true records, and a wrong link there wipes out a full point, so the bar must be high. Once a business already has an accepted link, it is very probably a real, active business, so a wrong extra link only costs part of a point, while a right one completes its list; the bar can come down a little. But when a business already has three or more links, one more true link adds little and a wrong one still costs a lot, so the bar goes back up. The **per-business consistency check** applies exactly this idea, with values tuned on held-out folds 8 and 9.

REMEMBER

- Bars 0.75 / 0.68 / 0.72 / 0.77 for 0 / 1 / 2 / 3+ other accepted records; lead 0.4; US and India only.
- Reason: the value of one extra link under per-business F0.5 depends on how many links the business has.
- Small effect: US +277 / -284, India +247 / -248; tuned on folds 8 and 9.

Training and validating without leaks

21. Data leakage (full: A3.3)

IN ONE LINE

Leakage is any route by which information about the answers, or about the evaluation data, reaches training and makes a model look better than it really is.

SIMPLE EXPLANATION

A student who has secretly seen the exam paper gets a great score, but the score says nothing about what the student knows. Leakage is any way the "exam paper" reaches the model. Sometimes it is obvious, like testing on the training data. More often it is sneaky: two records of the same business end up on both sides of the split; a feature is computed with help from the labels; one model's over-confident score on its own training data is fed to another model; or settings are chosen by looking at test results again and again. The symptom is always the same: an excellent validation score followed by a disappointing real score. Most of the fold design in this part exists to block one leakage route or another.

REMEMBER

- Leakage = the answers or the evaluation data reaching training by any route; the result is a score that will not hold.

- Our defences: business folds, separate folds per model layer, word lists learned without the evaluated folds, removal of records touching the scored fold.
- Unsupervised statistics on the unlabelled test files are transductive, allowed and disclosed; no test label is used.

22. Group k-fold: folds by business (full: A3.4)

IN ONE LINE

Every incoming record must sit in the same fold as its business, so the models are always tested on businesses they have never seen.

SIMPLE EXPLANATION

Imagine teaching a child to recognise relatives from photographs. If you test with a different photo of the same uncle the child practised on, the child may simply remember that uncle's face rather than learn a general skill. Our "uncles" are businesses: each business owns several incoming records (about 5.75 on average in the test files). If records were split at random, some records of a business would be in training and others in testing, and the model could lean on memory of that particular business, its name, its address, its competitors. **Group k-fold** keeps the whole family together: a business and all its records go into the same fold. Then a good test score really means "works on new businesses", which is what the competition asks.

REMEMBER

- The business is the unit of the metric and of the collective features, so it must be the unit of the split.
- A record-level split would put records of one business on both sides and inflate the score.
- Fold sizes came out balanced: fold 9 US 132,365 vs fold 8 US 132,407.

23. The held-out evaluation protocol (full: A3.8)

IN ONE LINE

To grade a recipe honestly on fold 9, we retrain it without fold 9 and without any incoming record that touches fold 9, and then compute the exact competition metric on fold 9's businesses (and the same for fold 8).

SIMPLE EXPLANATION

Think of re-running a whole training camp without the players you want to test, so nobody in the camp has practised against them. There is one extra twist. An incoming record can have candidates in several folds: for example, its true business is in fold 9 and a look-alike candidate is in fold 7. If that record stayed in training through its fold-7 pair, the model would learn "this record is not the fold-7 business", which partly reveals the fold-9 answer by elimination, and the competition features that compare the two candidates would tie them together. So we remove from training every incoming record that has any candidate in the fold being scored. That is the meaning of "held-out" in our numbers.

REMEMBER

- Held-out = refit without the scored fold and without every record touching it, fixed tree count, exact per-business F0.5.
- Fold 9 / fold 8: US 0.991212 / 0.991545; India 0.990046 / 0.990298 (3 of 5 seeds).
- It excludes the second-chance searches and the France rules, and folds 8 and 9 also chose thresholds.

24. Early stopping, then refit on more data (full: A2.24)

IN ONE LINE

We first find the right number of trees on a validation fold, then retrain on more folds (6-9) with that number scaled up in proportion to the extra rows.

SIMPLE EXPLANATION

A recipe tested for four people tells you the cooking time; when you cook for eight, you adjust the time for the bigger batch instead of guessing. Early stopping on fold 8 tells us how many trees suit a model trained on folds 6-7. When we retrain on folds 6-9, roughly twice the rows, no fold is left to stop on, so we scale the number of trees by the ratio of rows.

WORKED EXAMPLE (TOY NUMBERS, THEN REAL PUBLIC SCORES)

Seed 1 is fitted on folds 6-7 with 1,000,000 rows and early stopping on fold 8 finds the best iteration at 600 trees. Folds 6-9 have 2,000,000 rows. The refit trains $600 \times 2,000,000 / 1,000,000 = 1,200$ trees with no early stopping. Because folds are assigned by a hash of the business id, folds are of similar size and the ratio is close to 2 (reasoning, not a reported number). Real: adding the US/India refit moved the public score from 0.990621 to 0.990676, and it was kept; the same refit for France scored 0.990726 against 0.990748 without it, and it was not adopted (Doc B.6).

REMEMBER

- Early stopping on fold 8 finds the trees; the refit on folds 6-9 scales them by $\text{rows}(6-9) / \text{rows}(6-7)$.
- US/India refit kept (public 0.990621 to 0.990676); France refit dropped (0.990726 against 0.990748).
- The refit is one reason there is no untouched end-to-end validation.

25. Seed averaging (full: A2.25)

IN ONE LINE

We train the same decision model five times with different random seeds and average the five probabilities, which smooths out the randomness of each individual fit.

SIMPLE EXPLANATION

Ask five equally trained judges, each of whom happened to study a slightly different random sample of the evidence, and take their average opinion: one judge's odd reading of a case matters less. In our LightGBM settings, each tree sees a random 80 % of the rows and 90 % of the features, so two fits with different **random seeds** (the starting number of the random-number sequence) give slightly different models.

WORKED EXAMPLE (TOY NUMBERS)

One candidate gets probabilities 0.78, 0.74, 0.71, 0.80 and 0.77 from five seeds. Their mean is 0.76, above the 0.75 threshold, so it would be linked if it is also a clear winner. Taken alone, two of the seeds (0.74, 0.71) would have rejected it. Variance view: if each seed's score varies with standard deviation σ and the seeds are correlated with $\rho = 0.5$, the variance of the five-seed mean is $\rho \times \sigma^2 + (1 - \rho) \times \sigma^2 / 5 = 0.5 + 0.1 = 0.6 \times \sigma^2$, a 40 % reduction.

REMEMBER

- Five seeds, same recipe, different random rows and features; average the probabilities.
- Reduces variance, not bias; the default route uses a single fit.
- Effect not isolated: it shipped together with the French E5 reader (0.991114 to 0.991300).

26. Self-training with pseudo-labels and replay (full: A2.21)

IN ONE LINE

When a country has no labels, a model's own very confident predictions on that country's records are used as training labels (pseudo-labels), mixed with real labelled examples so the model does not forget what it already knew.

SIMPLE EXPLANATION

A teacher moves to a school in another country with no answer keys. She marks only the papers she is completely sure about and uses those to adjust her marking to the new style; that is **self-training**, and her confident marks are **pseudo-labels**. To avoid drifting into her own mistakes, she keeps re-marking some old papers whose answers she knows for certain; that is **replay**. She also sets aside a few of her confident marks, never uses them for practice, and later checks that she still agrees with them.

WORKED EXAMPLE (DERIVED: THE DOCUMENTED FRENCH E5 SELECTION RULE APPLIED TO THE REAL LOGITS OF DOC D.3)

The rule: confidence p = mean of sigmoid(MiniLM logit) and sigmoid(French MiniLM logit). A pair becomes a **positive** if it is the record's best pair, $p \geq 0.97$, and every other pair of that record has $p < 0.5$. The record's highest- p pair with $p \leq 0.03$ becomes a **negative** (the hardest confident non-match).

1. TQ Parents SARL (same address): $p = (0.99945 + 0.99987) / 2 = 0.99966$, at least 0.97.
2. TQ Comite SARL (Pessac): $p = (0.99920 + 0.12765) / 2 = 0.5634$, which is not below 0.5.
3. So the record has no positive (its runner-up is not clearly rejected) and no negative (no pair at or below 0.03): it contributes nothing. The selection is deliberately strict.
4. Real totals (Doc 4, B.4): 474,808 French pseudo-labelled pairs plus 474,808 labelled US/India pairs, 949,616 in all; 5 % of French records held out, giving 25,192 held-out pairs (12,616 positive).

REMEMBER

- Pseudo-labels: only very confident pairs (p at least 0.97 with a clearly rejected runner-up; negatives at most 0.03).
- Replay: about one to one with labelled US/India pairs (exactly 474,808 + 474,808 for the French E5 reader).
- Checks: two passed, two failed; reader kept on public score; no measured French F0.5.

27. Synthetic training pairs (full: A3.19)

IN ONE LINE

Synthetic pairs are labelled examples you create yourself instead of collecting them, used to teach or test a model on situations that are rare or unlabelled in the real data.

SIMPLE EXPLANATION

Pilots practise engine failures in a flight simulator because real failures are rare and dangerous. Synthetic training pairs are simulator practice for a matching model: cheap, plentiful, and aimed at exactly the situations you care about. The catch is that a simulator is never perfectly realistic. A model can learn simulator habits that do not occur in reality, or learn the wrong proportions of easy and hard cases. So synthetic data must always be checked against real labelled data, and the amount must be controlled. In this competition synthetic pairs were explicitly allowed (external data were not).

WORKED EXAMPLE

(toy textbook illustration of the general technique; this is not how our own extra examples were made)
From one real labelled match, "Green Leaf Cafe, 5 Park Street, Kolkata" ↔ its business:

Created pair	How	Label
"Green Leaf Cafe, 5 Park St, Kolkatta"	abbreviation and a typo	match
"GREEN LEAF CAFE, PARK STREET, KOLKATA"	upper case, house number dropped	match
record ↔ "Green Valley Cafe, 12 Camac Street, Kolkata"	paired with a different real business of the same city and trade	non-match

Then the essential check: train once with and once without the created pairs, and compare on real held-out labelled data. If the real held-out score does not improve, the synthetic pairs are not helping, however good they look.

REMEMBER

- Synthetic pairs were allowed; external data were banned and not used.
- We used synthetic examples in two places: extra hard training examples for US/India decision models, and a generated labelled French set for evaluation only.
- Always validate synthetic-data choices on real labelled data; a synthetic test can disagree with reality.

28. Weak supervision and rules computed by code (full: A3.23)

IN ONE LINE

Weak supervision means using programmatic knowledge (rules, heuristics, word lists) instead of hand-labelled examples when labels are missing; our France rules are a related but simpler idea: fixed rules, written as code, that veto or rescue links after the decision model.

SIMPLE EXPLANATION

With no answer key, an expert can still write rules of thumb such as "if the record is at the same address as the business but one word of the name was swapped for a common category word like 'comité', it is probably a different organisation". In classic weak supervision (for example the Snorkel system), many such rules vote on unlabelled examples, a statistical model estimates how reliable each rule is, and the resulting noisy labels train a normal model. We did something simpler and more direct for France: each rule is a piece of code that reads the decision model's scores, the raw text and the address, and either removes a link (a **veto**) or adds one (a **rescue**). No human or AI judges any pair when the pipeline runs.

REMEMBER

- Our France rules are deterministic code on scores, raw text and address: vetoes and rescues after the decision model.
- France links: 887,630 accepted → 867,559 after eight rules; the two word-swap vetoes remove 29,021.
- Limitation first: designed after analysing unlabelled French test records, some parameters chosen after public scores; disclosed.

29. Domain shift and unseen countries (full: A3.24)

IN ONE LINE

Domain shift is when the data a model meets in use differ from the data it learned from; a country the model has never seen is the extreme case.

SIMPLE EXPLANATION

A doctor trained in Chennai who moves to Paris knows medicine, but the forms, the abbreviations, the names and even how common each illness is are different. Our models learned on US and Indian records. France has different address formats ("15 bis", "R" for "Rue", postcodes, department names), different legal words (SARL, SAS, EURL, SCI) and, crucially, no labels. There are three kinds of change to watch: what the inputs look like (**covariate shift**), how common matches are (**prior** or **label shift**), and what counts as a match for a given input (**concept shift**). Our French plan answers each one: clean the French format into a familiar shape, let readers adapt themselves on French text, use a stricter bar, and add careful rules. For a country we have never seen at all, there is a cautious default route.

WORKED EXAMPLE

(toy) Covariate shift in the address:

Raw French address	Problem for a US/India-trained model	After French cleaning (our rules)
"15B R DE LA PAIX, PARIS"	"15b" is an unknown token and "r" is not a recognised street type	"15 bis", and "R" and "Rue" mapped to one street-type form, so the number feature sees 15 and the street words line up

(toy) Prior shift: suppose 60 % of training records have a true business but only 40 % do in a new country. A threshold tuned where matches are common produces too many links where they are rare, because more of the borderline candidates are wrong there. A stricter threshold is the simple defence.

(ours) How much France matters: our diagnostic upload that emptied every French row dropped the public score from 0.970125 to 0.839.

REMEMBER

- Three shifts: inputs (covariate), match rate (prior), meaning of a match (concept).
- France: French cleaning, self-trained French readers, US/India-trained decision model, stricter 0.82, rules by code; no measured French F0.5.
- Unseen country: default route (MiniLM reader only, single-fit decision model, 0.82 / 0.4), untested on real data.

30. Model selection and public-leaderboard overfitting (full: A3.25)

IN ONE LINE

Every time you choose between versions by looking at the same evaluation scores, you fit those scores a little; after many uploads, the best public score is biased upwards.

SIMPLE EXPLANATION

Flip 18 ordinary coins ten times each and pick the one that showed the most heads: it looks lucky, but it is an ordinary coin, and it will not keep winning. Leaderboard scores contain noise too, because each is computed on one particular set of test records. If you upload many versions and keep the best, you reward both real improvements and luck. This is **adaptive overfitting**, also called the **winner's curse**. The defences are: decide mainly with your own held-out folds, treat tiny public differences as ties, limit the number of uploads, and be open about which decisions used the leaderboard.

REMEMBER

- Picking the best of many noisy scores inflates it (about 1.8σ for 18 equal versions).
- Held-out folds set the thresholds; public scores chose the refit, the dropped variants, some France parameters and the final file.
- Mitigation: held-out scores in the same range; late public gains small (0.991114 → 0.991300).

4. The 40 most likely questions

The 40 most central of the questions the full guide tags **LIKELY**, in the order and with the numbers of the full guide. Drill them: read the question, answer aloud in 20-30 seconds, then compare with **Say**. The numbers and sources behind each answer are under the same Q-number in the full guide.

Category 1: Problem and data

Q1. What exactly is the task, in one sentence? ★ **LIKELY**

Why they ask: They check that you can frame the problem precisely before talking about models.

SAY We link each incoming record, from Source 2 or Source 3, to at most one business in the reference list, or to none, and one business may own several incoming records. All matching happens inside one country, and the test set has about 10 million incoming records against 1.7 million businesses.

IF THEY PUSH It is many-to-one record linkage against a reference list, not clustering: there is no record-to-record graph to close, so the natural decision is "which business, if any, for this record". The score is computed per business, and both output files have one row per business.

Q3. What makes this problem hard? ★ **LIKELY**

Why they ask: They test whether you understood the real difficulty, not just the mechanics.

SAY Three things. Scale: all same-country pairs would be 6.7 trillion. Look-alikes: many businesses share almost the same name, so the decision is between near-identical candidates, and "none of them" must always be an allowed answer. And France came with no labels at all.

IF THEY PUSH On top of that the text is noisy and multilingual: records without an address, abbreviations, names in Devanagari, French conventions such as "15B" or "R" for "Rue". Our India example shows the look-alike problem: three businesses all called "Vijay Enterprises Private Limited", in Agra, Satara and Byculla.

Category 2: Metric and the precision-first choice

Q16. How is the competition metric computed? ★ **LIKELY**

Why they ask: Everything else follows from the metric; they check you can state it exactly.

SAY For each business we compare the incoming records we linked to it with its true records and compute F0.5, then we average over all businesses, including those with no true records. A business with no true records scores 1 only if its row is empty.

IF THEY PUSH $F0.5 = 1.25 \cdot P \cdot R / (0.25 \cdot P + R)$, which per business simplifies to $1.25 \cdot TP / (\text{predicted} + 0.25 \cdot \text{true})$, and is 1 when both counts are zero. Illustration with made-up counts: 4 true records, we link 3 of them and nothing wrong, so $1.25 \cdot 3 / (3 + 0.25 \cdot 4) = 0.9375$.

Q18. Why does the empty-row rule matter so much? ★★ LIKELY

Why they ask: This rule is the sharpest edge of the metric; they test whether you designed around it.

SAY Because a business with no true records gets a full point only if we link nothing, so one wrong link turns its 1 into 0, the largest loss possible for one business. A correct empty row is worth as much as a perfect match list, and 100,087 of our rows are empty.

IF THEY PUSH With macro averaging these businesses weigh exactly as much as busy ones, so a liberal threshold would be punished hardest here. It is one reason for the clear-winner rule, the stricter French threshold and France rules that remove more links than they add.

IF THEY PUSH FURTHER Empty rows: US 38,161, India 46,681, France 15,245. Each empty row scores either one (if the business truly has no records) or zero (if it had some), and we cannot tell which without labels.

Q21. How did you choose the decision threshold, and why not tune for AUC? ★★ LIKELY

Why they ask: They test whether operating points were set on the right objective and on held-out data.

SAY We tuned it directly on the competition metric, exact per-business macro F0.5, on held-out fold 8, which gave 0.75 for the US and India. AUC measures how well all pairs are ranked and has no threshold in it at all, while our decision is at most one link per record, scored per business with empty rows worth 1, so the best threshold can only be read off that exact metric.

IF THEY PUSH The per-business consistency check was tuned on folds 8 and 9 and the second-chance thresholds on fold 8 with false positives counted twice. A real example of AUC and F0.5 disagreeing: on our generated French test set the French E5 reader had the higher AUC, 0.98038 against 0.979415, but the lower best F0.5, 0.910516 against 0.914964.

IF THEY PUSH FURTHER The exact metric matters because averaging per business gives small and empty businesses the same weight as large ones, so the best pair-level threshold is not the best business-level one. The honest caveat: fold 8 also appears in our held-out table, so that table is not an untouched test of this choice, as our documentation states.

IF THEY PUSH FURTHER Per business, $F0.5 = 1.25 \cdot TP / (\text{predicted} + 0.25 \cdot \text{true})$. A business with 4 true records scores 1.0 if all are found, 0.9375 if one is missed, and 0.833 if all are found plus one wrong link, so here a wrong link costs about 2.7 times a miss; at a business with no true records a wrong link costs the whole point. The E5 readers' AUCs are about 0.9985, which says nothing about this trade-off.

Category 3: Normalisation and multilingual text

Q31. How do you handle names written in Indian scripts? ★ LIKELY

Why they ask: Multilingual input is a stated challenge; they want the mechanism.

SAY In two layers. During cleaning, each native-script name word is mapped to Latin with a word list learned from labelled training matches, with a generic transliteration as fallback; and for records still unlinked, the native-script second-chance search tries Latin spellings of the name within the same state, which added 9,759 links in India.

IF THEY PUSH In our India example the second-chance search's list maps विजय to vijay, एंटरप्राइजेज to enterprises, प्राइवेट to private and लिमिटेड to limited; without the legal words the key is (vijay, enterprises), searched within Maharashtra. The readers also read the Devanagari directly, because their multilingual tokenizer covers it.

IF THEY PUSH FURTHER The cleaning word list is a co-occurrence alignment learned on folds 0-5: a native token gets a Latin word if they co-occur at least 3 times, with purity at least 0.35 and a score at least 1.8 times the runner-up; otherwise unidecode is used. It is not a neural transliterator. The second-chance list keeps every alternative with at least 5 % of a token's count and tries up to 64 transliterations. Separately, a label-free word alignment on test records whose top candidate has probability at least 0.9 feeds some name features; our documentation discloses it.

Category 4: Blocking and candidate generation

Q45. Walk me through your candidate funnel. ★ LIKELY

Why they ask: They want the count at each step and to see where cost and recall go.

SAY Four steps, from cheap to expensive: the inverted index finds about 130 candidates per business, a quick rule-based filter with no model drops the clearly weaker ones, the CatBoost ranker keeps at most four per incoming record, and the two second-chance searches add a few for records left unlinked. We end at about 7 per business.

IF THEY PUSH In numbers: 130.1 per business in our 25 September blocking study, 21.7107 after the filter (37,614,782 pairs), 6.75 in the shortlist (11,687,317 pairs, derived) and 7.0954 in the final file (12,293,019 pairs). Per incoming record that is 1.2331 candidates, and 90.59 % of records have exactly one.

Q53. How do you know blocking does not lose true matches? ★ LIKELY

Why they ask: Recall is the hidden ceiling of every entity-resolution system; they want measured evidence.

SAY We measure recall at every candidate step on labelled fold 9: the index keeps the true business for 99.37 % of US records and 98.51 % of Indian ones, and the shortlist loses about half a percentage point more, mostly to the cap of four. The quick filter is almost free.

IF THEY PUSH In the fold-9 samples the filter moves US recall from 99.3658 % to 99.3523 % and India from 98.388 % to 98.3745 %; the shortlist's full-fold lower bounds are 98.8748 % and 98.0572 %. These numbers use a ranker that never saw fold 9 and exclude the second-chance candidates, which target the remaining miss types: native-script names, and the same address with a different name.

Category 6: The CatBoost ranker and the shortlist

Q72. Why at most four candidates per incoming record? ★★ LIKELY

Why they ask: Every cap is a trade-off; they want both the cost side and the recall side, with a measurement.

SAY It is a cost-recall trade-off, and it is our largest recall loss, which we measured rather than assumed. On held-out fold 9, the index finds the true business for 99.37 % of US records and the shortlist still holds it for at least 98.87 %. In return, the readers see on average little more than one pair per record instead of the three to four that survive the filter.

IF THEY PUSH For India the same step goes from 98.51 % to at least 98.06 %. The cap bounds the transformer passes per record, which keeps cost linear; at larger scale we would shard the index by state or postcode to afford a larger cap where it loses owners, and the second-chance searches recover part of what the index and the cap miss.

IF THEY PUSH FURTHER Ranks 2 to 4 are kept only if the ranker gives them at least 0.001, so the shortlist averages 1.17 pairs per record (derived), not four. For India the same step goes from 98.5068 % to 98.0809 %, and the second-chance searches exist to win back the miss types the index and the cap leave behind.

Category 7: The readers (cross-encoders)

Q88. If the readers are that good, why do you need a decision model? ★★ LIKELY

Why they ask: This tests whether you see matching as a comparative decision rather than a pair classification.

SAY A reader scores one pair at a time, but the decision is a competition: among a record's candidates and, for the US and India, among records competing for the same business. The decision model sees the reader logits next to the ranker, each candidate's rank and lead within the shortlist, the "what changed" features and the collective features, and gives one probability on which we set an operating point for the exact metric. A reader also cannot see corpus-level facts, such as how rare a dropped word is across the country.

IF THEY PUSH Transformers are unreliable at arithmetic on digit strings, so exact number differences come in as features. Even with pair-level AUC near 0.999, a record can have two plausible candidates, and choosing between them, or abstaining, needs the comparison that the decision model's context features and the lead rule provide.

Category 8: Features and the "what changed" idea

Q95. What are the "what changed" features, in plain words? ★ LIKELY

Why they ask: This is your headline idea; they check you can explain it simply and precisely.

SAY For every candidate we describe the edit that turns the business into the incoming record, not only how similar the two are. Did a number change, and by how much? Which words were added, dropped or replaced, how rare are they and how close in meaning, and did the legal form change? The decision model then learns from labels which edits are harmless.

IF THEY PUSH There are 25 generic pair features: 8 for numbers, 14 for words and 3 for meaning similarity, plus a legal-form relation (same, changed, record has none, business has none, superset, subset, both none) and a number relation (none, same, extra only, dropped only, one side without a number, substituted).

Q96. Why describe differences instead of just measuring similarity? ★★ LIKELY

Why they ask: They want the reasoning behind the innovation, not only its description.

SAY A similarity score treats two edits of the same size as equal, but they mean different things. Studying the labelled data, we saw that true records of a business tend to lose information: they drop a digit or a suffix, abbreviate, or transliterate. So we describe the edit itself and let the model learn from labels which kinds of change are harmless and which point to a different business.

IF THEY PUSH "Legacy Marketing Enterprises" against "Legacy Marketing Industries" is very similar by characters, yet it is an in-place replacement of one distinctive word, and the decision model gives it 0.000088. Because the features encode operations rather than word lists, the same definitions carry over to another country.

Q102. Did the "what changed" features actually help? Do you have an ablation? ★★★ LIKELY

Why they ask: Claims of novelty need evidence; they test your honesty about what was and was not measured.

SAY We did not run a clean ablation that removes all "what changed" features. The closest evidence: when we replaced data-specific inputs with the final generic "what changed" inputs, the public score moved from 0.990881 to 0.990934, so the generic version cost nothing and helped slightly. A code check asserts that none of the removed inputs reaches a final decision model.

IF THEY PUSH On held-out folds the generic version was within a few hundred-thousandths of the earlier one, so we chose it for generality, not for a measured gain. The ablation we would run: retrain the decision models without the 25 features, score folds 8 and 9 with the exact per-business F0.5, and report a paired bootstrap interval over businesses.

Category 9: Decision models and thresholds

Q110. Explain the "confident and clear winner" rule. Why do you need both conditions? ★★ LIKELY

Why they ask: The decision rule is where precision is won or lost; they test the logic.

SAY Per incoming record we take the best-scoring candidate and link it only if its score is at least 0.75 and it leads the runner-up by at least 0.4; otherwise we link nothing. The threshold asks "is it likely enough?", the lead asks "is it clearly better than the alternative?", which protects us against near-ties between two plausible businesses, where picking either one risks a wrong link. If a record has only one candidate, the lead condition passes automatically.

IF THEY PUSH It is a reject option, arg-max with an absolute threshold and a margin, and at most one link per record enforces the many-to-one structure. Why abstaining pays under F0.5, as an illustration with per-business $F0.5 = 1.25 \cdot TP / (\text{predicted} + 0.25 \cdot \text{true})$: for a business with 3 true records, linking all 3 plus 1 wrong one scores $3.75 / 4.75 = 0.79$, while missing one and linking only 2 correct ones scores $2.5 / 2.75 = 0.91$, so a wrong link costs more than a missed one.

Q111. How did you choose the 0.4 lead? ★★★ LIKELY

Why they ask: They noticed one parameter shared by every country and want to see whether it was justified.

SAY 0.4 is one fixed lead used in every country's rule, and the US and India threshold next to it was tuned on fold 8 for the exact metric. Our documentation records the threshold choice but not a separate study of the 0.4, so we do not claim one. Its role is clear: with scores between 0 and 1, a lead of 0.4 means we link only when one candidate clearly dominates.

IF THEY PUSH Most records have a single candidate, where the lead passes automatically, so 0.4 acts on the minority with real competition. The evidence that would defend it with a number is a joint threshold-by-lead sweep on fold 8, reported on fold 9; our documentation does not include one.

Q113. Why are the decision models trained on folds 6-9, and how do fit, early stopping and refit work? ★★ LIKELY

Why they ask: They test the training protocol for leakage and how the number of trees was set.

SAY The ranker and readers learn on folds 0 to 5, so the decision models train on folds 6 to 9, where those scores are out of sample, just as on test. For the US and India each seed is fitted on folds 6 and 7 with early stopping on fold 8, then refitted on folds 6 to 9 with the best iteration scaled by the row count, since no fold is left for stopping. On the public board that refit moved 0.990621 to 0.990676; the same refit for France scored lower, so France keeps the folds 6-7 fit.

IF THEY PUSH The fit uses early stopping with patience 100 and at most 3,000 rounds (1,500 for France); the refit uses best iteration $\times$ rows(6-9) / rows(6-7) rounds, a heuristic based on the idea that more data supports more trees at the same learning rate. The France refit scored 0.990726 against 0.990748 without it.

Category 10: Second-chance searches and the consistency check

Q118. What are the second-chance searches, and why do you need them? ★ LIKELY

Why they ask: They test whether you diagnosed the type of error before adding a fix.

SAY Some misses are retrieval failures: the right business never entered the shortlist, so no threshold can fix them. Two targeted searches revisit records still unlinked after the main decision: a native-script search for Indian names, which added 9,759 links, and an address search for the US and India, which added 3,137. Both touch only unlinked records, so they never undo a link.

IF THEY PUSH Each targets a miss type we found on training data: native-script names whose transliteration the Latin index does not retrieve, and records whose name differs while the address matches. Their candidates join the candidate file, which grows from the 11,687,317-pair shortlist to 12,293,019 pairs: 578,317 native-script pairs plus 27,385 address pairs (derived).

IF THEY PUSH FURTHER The native-script search scored 649,965 pairs for 171,713 records, 578,317 of them not already shortlisted; the address search retrieves businesses sharing one of the record's 3 rarest digit runs and one of its 3 rarest alphabetic address words (3,029,652 US and 4,766,484 Indian pairs, scored by a filter), and only 1,827 US and 25,558 Indian new candidates survive. The file is then $11,687,317 + 578,317 + 25,558 + 1,827 = 12,293,019$ pairs.

Q125. What is the per-business consistency check? ★★ LIKELY

Why they ask: It is an unusual step; they want to know what it does and whether it is principled.

SAY After the main decision in the US and India, we re-decide each record with a bar that depends on how many other incoming records the decision model already accepted for the same business: the usual 0.75 with none, a slightly lower bar with one or two, and a slightly higher one with three or more, always with the same lead. The bars were chosen on folds 8 and 9, and the step moves only a few hundred links in each direction per country.

IF THEY PUSH The bars are 0.75, 0.68, 0.72 and 0.77 for zero, one, two and three or more other accepted records, with the 0.4 lead. Final run: US +277 / -284, India +247 / -248, with one addition skipped because that record already had a native-script link, since the check never adds a link to such a record. The count of other accepted records is computed once from the main decision, with no iteration.

Category 11: France with zero labels

Q128. France has no labels. How did you build it at all? ★ LIKELY

Why they ask: This is the hardest part of the problem; they want the overall strategy in one breath.

SAY Four pieces: French cleaning rules for addresses and legal forms; the decision model trained on labelled US and Indian rows, used at a stricter 0.82; two French readers self-trained on our own most confident French predictions, mixed with labelled US and Indian pairs; and rules computed by code that veto risky links and rescue safe ones. The index, quick filter, ranker and shortlist are the same as in the other countries.

IF THEY PUSH The France decision model has 158 features, is the mean of five seeds and is trained on labelled US and Indian rows; it accepts 887,630 French links and the rules leave 867,559. The plain E5 reader is not used for France; its slot is a blend of the MiniLM, French MiniLM and French E5 readers.

Q131. How did you set the French threshold of 0.82 without labels? ★★ LIKELY

Why they ask: An operating point without labels is a hard problem; they want the method and its weakness.

SAY Without French labels we cannot tune on French F0.5, so we set a more conservative point without labels: from the number of French links per business, together with a threshold sweep on US and Indian fold 8. It is stricter than the 0.75 we use where labels exist, because the France decision model is applied outside the data it was trained on.

IF THEY PUSH For context, France ends with 3.3438 links per business against 3.3935 (US) and 3.3817 (India), from 5.531 incoming records per business in the test files. The held-out proxy of the France recipe at this operating point is 0.990508 (US) and 0.989467 (India) on fold 9; with a small labelled French sample, this threshold is the first thing we would re-tune.

IF THEY PUSH FURTHER Mean links per business: France 3.3438, US 3.3935, India 3.3817. The France recipe at 0.82 on labelled US and Indian rows scores 0.990508 and 0.989467 on fold 9. Per incoming record, France links slightly more often (0.6046 against 0.5895 and 0.5806) because it has fewer incoming records per business (5.531 against 5.756 and 5.824); without labels we cannot tell whether that means more owned records or extra wrong links. Link volume is only a sanity check: it says nothing about whether the links are right.

Category 12: Training strategy

Q139. Which data trained which model? ★ LIKELY

Why they ask: Before trusting any score, a scientist checks exactly what each model learned from; it is the first test for leakage.

SAY We split the businesses into ten folds and gave each layer of the pipeline its own folds. The CatBoost ranker and the readers learn on folds zero to five. The LightGBM decision models use folds six to nine: they fit on six and seven, stop early on eight, and the US and India models are then refitted on all four. The two second-chance models have their own training sets, and the French readers learn by self-training on unlabelled French records, mixed with labelled US and Indian pairs.

IF THEY PUSH The full map: the Indic-to-Latin cleaning word list is learned on folds 0-5; the ranker on a 140,000-record pilot, of which the fold 0-5 part is fitted, with early stopping on fold 6; the MiniLM and E5 readers on folds 0-5 for one epoch; the US and India decision models are fitted on 6-7, early-stopped on 8, then refitted on 6-9; the France and default decision models are fitted on 6-7 and early-stopped on 8, with no refit; the native-script model on folds 0-7; the address second-chance models on folds 6-7. The 0.75 threshold was chosen on fold 8, the consistency-check thresholds on folds 8 and 9.

Q140. Why do the ranker and readers train on different folds from the decision models? ★★ LIKELY

Why they ask: The classic stacking trap: a model that sits on top of other models must never see their scores on records those models trained on.

SAY Because the decision model uses the ranker probability and the reader scores as inputs. If those models had trained on the same records, their scores there would be over-confident, and the decision model would learn to trust them more than it should on new data. So the ranker and readers learn on folds zero to five, and the decision models only ever see their scores on folds six to nine, which those models never trained on.

IF THEY PUSH This is stacked generalisation with a layer-wise split rather than out-of-fold predictions. Out-of-fold scoring would let the decision model use all ten folds, but it needs several copies of every reader and of the ranker; the layer split costs labelled data instead of compute. One precise caveat: the ranker used fold 6 for early stopping, so fold 6 is not perfectly untouched for it, although that choice sets only the number of trees. A useful side effect: the readers' quality on folds 8 and 9 is an honest held-out number, for example an AUC of 0.998570 for the E5 reader.

Q141. Why assign folds by business rather than by record? ★ LIKELY

Why they ask: Grouped splitting is basic hygiene for entity resolution; a random record split is the most common way teams inflate their scores.

SAY Because the business is the unit of the metric and of several features. A business owns several incoming records, so a record-level split would put records of the same business on both sides, and we would be testing on businesses the model had partly seen. We hash each business id into one of ten folds, and every record travels with its business.

IF THEY PUSH An incoming record that belongs to no business hashes its own id. The collective features and the per-business consistency check look at all records competing for one business, so a record-level split would leak through them as well. The held-out protocol is stricter still: when we score a fold, the fits also drop every incoming record that has any candidate in that fold.

Category 13: Validation, model selection and the leaderboard

Q149. How did you validate your models? ★ LIKELY

Why they ask: The single most important credibility question: is your offline evidence the real metric, on data the model never saw?

SAY With the exact competition metric, per-business macro F0.5, on labelled folds the models did not train on. To score fold nine we refit the decision models on folds six to eight, leaving out every record that touches fold nine, and the same for fold eight. Close versions of our decision models score between 0.990 and 0.992 there, the same range as our public 0.9913.

IF THEY PUSH US rows 0.991212 (fold 9) and 0.991545 (fold 8); Indian rows 0.990046 and 0.990298. Both come from the India recipe with 3 of its 5 seeds (the final US model trains on US rows only and adds 6 features), with the 0.75 threshold and the consistency check. They exclude the second-chance searches and the France rules, and the number of trees in each fit follows the same row-scaling rule as the final refit.

Q152. Which decisions used held-out folds, and which used the public leaderboard? ★★ LIKELY

Why they ask: They want an honest map of where offline evidence ended and leaderboard feedback began.

SAY Held-out folds decided the quick filter, the 0.75 threshold, the consistency check and the second-chance thresholds, and the French E5 reader was measured against written checks. Public scores decided the US and India refit, dropping the France refit, a second French self-training round and a retrained French MiniLM reader, some France rule parameters, and the final file among our uploads.

IF THEY PUSH Label-free counts on the test records set the France threshold 0.82 and the French MiniLM reader's blend weight. Public scores also calibrated the label-free estimates for transferring the address search to the US and for the generic-word rescue, which then selected two further families of that rescue. Keeping the French E5 reader went against its own written rule.

Q153. How did you avoid overfitting the public leaderboard? ★★★ LIKELY

Why they ask: Many uploads plus selection by public score is a known way to overfit; they test your honesty first, then your mitigation.

SAY Not completely, and we say so. Our US and Indian thresholds came from held-out folds, not from the leaderboard, but we made 18 uploads, picked the final file among them by public score, and let public feedback shape some France rule parameters. What limits the damage is that held-out scores of close versions of our models sit in the same range, and the late public gains were small.

IF THEY PUSH The last step was 0.991114 → 0.991300, a gain of 0.000186 (derived). Held-out: US 0.991212 / 0.991545, India 0.990046 / 0.990298. We also dropped three changes on public scores (the France refit, a second French self-training round and a retrained French MiniLM reader); to be clear, that is more leaderboard selection, not a safeguard against it. What limits it is size: those three differences were 0.000022, 0.000023 and 0.000042 (derived), so little could be gained or lost by each choice.

Q154. Will 0.9913 hold on the private leaderboard? ★★★ LIKELY

Why they ask: Tests whether you understand selection bias and can say where your risk sits.

SAY We can't promise it. There is some selection bias, because we chose the final file among our uploads using public scores, which pushes the public number up a little. Held-out scores of close versions of our models are in the same range and the late gains were small, which limits that; the part with the most risk is France, because it has no labels and its rules were tuned with public feedback.

IF THEY PUSH Held-out fold 9 / fold 8: US 0.991212 / 0.991545, India 0.990046 / 0.990298; they cover the main decision only. The French E5 reader, which failed two written checks, also sits in the French part.

Category 14: Edge cases

Q162. What do you do with a country you have never seen? ★ LIKELY

Why they ask: Generalisation beyond the test countries is part of the brief; they want a concrete fallback and honesty about testing.

SAY It takes a default route: the US and India cleaning rules, the multilingual MiniLM reader only, a decision model with 158 features that exist in every country, trained on labelled US and Indian rows, and the stricter 0.82 threshold, with no second-chance searches or rules. Honestly, it is untested on real data, because the test set had only the US, India and France.

IF THEY PUSH The index and its rarity weights are built per country from that country's own businesses, so retrieval needs no labels. The proxy row in our held-out table is this same decision-model recipe as a single fit, run on labelled US and Indian rows at 0.82: 0.990508 and 0.989467 on fold 9; but those are countries it was trained on, not a new one. For a real new market we would add a country normaliser, self-train readers as for France, and label a small sample to set the threshold.

Category 15: Results and error analysis

Q172. What is your final result? ★ LIKELY

Why they ask: The opener; they want the number and whether it is backed by offline evidence.

SAY Our final file scores 0.9913 F0.5 on the public leaderboard, up from 0.968 for our first scored baseline. On held-out folds, close versions of our US and Indian decision models score between 0.990 and 0.992, so the public number is in line with what we measured offline.

IF THEY PUSH Exactly 0.991300, upload 16 of 18. The file has 5,856,936 links and 12,293,019 candidate pairs (7.10 per business), and it passes the organisers' validator.

Q174. What are your most common wrong links? ★★★ LIKELY

Why they ask: Error analysis shows whether you understand your model's failures, which matters more than the score.

SAY Three kinds: names at the business's own address with one generic word replaced or added or the legal form changed; acronyms that differ from the business name by one initial at another address; and branches of a chain at nearby addresses. The "what changed" features, the clear-winner rule and, in France, the vetoes each target one of these.

IF THEY PUSH In France the two word-swap vetoes removed 29,021 of 887,630 accepted links (3.3 %, derived), the street veto 826, the generic-word swap veto 538 and the cell veto 1,322. In our US example the legal-form change from Inc to LLC is scored down to 0.000007, although the ranker had put it second.

Category 16: Limitations and what we would do differently

Q181. What is the biggest limitation of your work? ★★ LIKELY

Why they ask: Senior scientists trust teams that name their own weak points before anyone else does.

SAY We never had a fully untouched end-to-end test of the final file. Our held-out scores cover the main decision models, but they exclude the second-chance searches and the France rules, the same folds also helped choose thresholds, and France has no labels at all. The mitigation is that held-out scores of close versions of our models sit in the same range as the public score.

IF THEY PUSH One more detail from our documentation: the final US and India decision models are refitted on folds 6-9, so the shipped models are not the exact ones scored. Held-out fold 9 / fold 8: US 0.991212 / 0.991545, India 0.990046 / 0.990298.

Q182. The France rules were designed by looking at the test set. Isn't that overfitting? ★★★ LIKELY

Why they ask: The most adversarial question you can get; they want the admission first, then the safeguards.

SAY Partly, yes, and we disclose it. The rules were designed after analysing the unlabelled French test records, and some parameters were chosen after seeing public scores. What limits the damage is that each rule is code on model scores, raw text and the address, with no test label and no human or AI judgement of any pair as an input; the largest action, the two word-swap vetoes, removes 29,021 of 887,630 accepted French links.

IF THEY PUSH The evidence for the rules is label-free: control groups on the French records, and for the cell veto a check on US labels that name noise and address noise are independent. The analysis included AI-assisted review of sampled pairs, about 1,000 pairs in five samples plus smaller checks. In production we would turn the rule conditions into features and learn them from a small labelled French sample.

Q183. Why did you keep the French E5 reader when it failed two of its checks? ★★★ LIKELY

Why they ask: They test whether you follow your own rules, and whether you are honest when you did not.

SAY Because the file containing it had our best public score: a leaderboard-driven decision against our own written rule, and we disclose it as such. It passed the US and Indian quality check and agreed 99.92 % with held-out French pseudo-labels, but it was slightly worse than its base model on our generated French test set and failed our rule that the expected gain must be safely positive. It changes French links only, and the same upload also added the five-seed models, so its own effect is not isolated.

IF THEY PUSH Generated French test set: best F0.5 0.910516 against 0.914964, although the blend that includes it beat the blend without it, 0.90843 against 0.898031. Gain rule: point estimate +0.0000302, pessimistic -0.0001806, optimistic +0.0002315, and the rule required the pessimistic estimate to be above zero. It changed French links by -2,228 / +3,396 (5,437 business rows); US, India and candidates were identical.

IF THEY PUSH FURTHER It passed the US/Indian quality tolerance (AUC 0.998544 against 0.998570) and agreement with held-out French pseudo-labels (0.999206 against a 0.98 bar). The gain rule's pessimistic estimate was -0.0001806; it changes French links only (-2,228 / +3,396), and the same upload also added the five-seed models, so its own public effect is not isolated.

Q192. What would you do differently? ★★ LIKELY

Why they ask: The reflection question; they want specific, prioritised lessons tied to your own limitations.

SAY First, keep an untouched end-to-end validation fold, including the second-chance searches, from day one. Second, add finer blocking keys such as state or postcode, which gives room for a larger cap than four where the cap loses true businesses. Third, label a small French sample, to replace the hand-written rules with learned features and set the French threshold on real labels; and fourth, fold the second-chance searches into one decision model.

IF THEY PUSH Also: a dense retrieval channel for brand and "doing business as" names, clean ablations of the "what changed" features and the seeds, and a single-record latency measurement.

Category 17: Why not X? Alternatives we did not use

Q194. Why not simply ask a large language model whether two records match? ★★ LIKELY

Why they ask: They want to see that small models were a deliberate choice (cost, control, competition rules), not a default.

SAY No LLM runs anywhere in our pipeline. The rules cap every model at 8 billion parameters, and even after blocking we score 11.7 million shortlist pairs, so a generative call per pair would cost far more than our readers, which have about 118 and 278 million parameters. Small fine-tuned readers plus a decision model that compares all candidates of a record were accurate enough: close versions of our decision models score between 0.990 and 0.992 on hidden folds.

IF THEY PUSH Our decision is a competition between up to four candidates with a tuned threshold and a 0.4 lead, so it needs a numeric score for every candidate; an LLM judge could give one through its token probabilities, but at the price of a forward pass through a model tens of times larger than our readers for each of 11.7 million pairs. During development, AI assistants helped write code and review sampled pairs when we designed the France rules, which we disclose, but no human or LLM judgement of any pair is an input to the pipeline. In production, an LLM could earn a place offline, for example pre-screening the uncertain band before human review, judged against a labelled audit sample like any other model.

Q195. Why an inverted index for blocking, and not embedding (vector) search with an approximate-nearest-neighbour index? ★★ LIKELY

Why they ask: Dense retrieval is the modern default; they test whether we considered it and know how we would judge it.

SAY We did not benchmark a dense retriever, so we cannot claim it would be worse. Our index keeps the true business for 99.37 percent of US and 98.51 percent of Indian held-out records, it is deterministic, cheap and easy to audit, and its separate lists for name words, address words, letter pieces and sounds already catch typos and abbreviations. A vector index would be a natural extra channel for brand or "doing business as" names that share few words with the business.

IF THEY PUSH In our view, embeddings are weakest exactly where our hard cases are: two names that differ only in the legal form or a single word get almost the same vector, so a dense channel would add recall for paraphrase-like names, not precision. We would add it as an extra candidate list unioned with the index output, train it contrastively with hard negatives mined from our own index, and judge it by the same per-stage recall table on fold 9, by candidates per business and by held-out F0.5, because extra candidates compete for the same four shortlist places.

IF THEY PUSH FURTHER Rare exact tokens such as house numbers and unusual name words are what sparse matching captures best, and its misses can be explained term by term. We would add dense retrieval as a sixth channel and measure the recall gain at each step before paying for the embeddings and an approximate-neighbour index.

Q200. Why separate models per country rather than one global model? ★ LIKELY

Why they ask: A global model is simpler to run; they check that per-country models are justified, not accidental.

SAY The countries differ in format, French addresses and legal words and Indian scripts, and in labels: France has none. The code path is the same for every country; a country profile only selects the cleaning rules, readers, decision model, threshold, second-chance searches and rules. We share where it helps: the India decision model and the France decision model both train on US and Indian rows.

IF THEY PUSH One model with a country feature could share more, but the operating points would still be per country: France, with no labels to tune on, uses a more conservative 0.82, against 0.75 tuned on fold 8 for the US and India. Per-country models also let us retrain or roll back one country without touching the others; the closest thing we have to a global model is the unseen-country route, trained on US and Indian rows with the MiniLM reader only, which is untested on real data.

Category 18: Taking it to production at Amazon scale

Q208. Amazon has billions of records. What would the deployed system look like? ★★ LIKELY

Why they ask: The core production question: does the design scale, and where would it break?

SAY The work per record is bounded at every stage, posting-list caps in the index, a fixed number of top candidates per list and at most four pairs per record for the readers, so cost grows linearly and shards run independently. Today we split by country; at billions we would split by country and state or postcode, keep each shard's index in memory and run the readers on a pool of GPUs. As a rough linear extrapolation, not a measurement, a billion incoming records means about 1.17 billion reader pairs, on the order of 140 GPU-hours spread across shards.

IF THEY PUSH Finer shards lose recall when the state is missing or wrong, so we would keep a fallback to the country-wide index, as our native-script search already does: it searches the record's state and falls back to all states when the state is unknown or has no match. The posting-list caps are absolute numbers: a word whose list is longer than the cap is skipped, and in much larger shards more words cross that line, so we would re-measure recall per stage on a labelled sample before trusting the extrapolation.

Q210. What is the latency for a single incoming record? ★★ LIKELY

Why they ask: Online matching needs a latency budget; they check whether we know ours and are honest if we do not.

SAY We have not measured single-record latency; our runs were batch jobs. The path per record is short: one index lookup in its shard, the ranker over a few filtered candidates, on average 1.17 reader pairs, and a handful of small tree-model calls per pair, so streaming is plausible in principle. The second-chance searches only run for records left unlinked and could run asynchronously.

IF THEY PUSH In batch, the index lookup with the filter and ranker took about 1.3 hours and reader scoring about 1.4 hours for 9.97 million records on one machine. For an online service we would measure p50 and p99 per stage; the reader is the stage to watch, because a cross-encoder pass cannot be precomputed per business.

IF THEY PUSH FURTHER For streaming we would keep the index and the business-side statistics in memory and refresh the rarity weights periodically rather than per record; the collective features and the per-business consistency check would need each business's current set of competing and accepted records. Those are design ideas, not things we built or timed.

Category 19: Reproducibility, engineering and rules

Q226. Did you use the test data in any way? ★★★ LIKELY

Why they ask: The key integrity question; they want a complete disclosure without having to drag it out.

SAY Yes, without labels, and it is all disclosed. The pipeline computes unsupervised statistics on the test files, such as word rarity, the search indexes and competition features and, for France, the geography and a street-type typo table, and it self-trains the two French readers on confident pseudo-labels of French test pairs. The France rules were designed after analysing the unlabelled French test records, including AI-assisted review of sampled pairs, and their parameters were chosen after seeing test results, including public scores.

IF THEY PUSH Label-free counts on the test records also set the French threshold, the French MiniLM reader's blend weight and, for the US and India, how many extra hard training examples to add; for India, a native-script word alignment is computed on test records whose first-stage top candidate has probability at least 0.9. During development, a generated labelled French test set made from the French test businesses was used for evaluation only. We never had test labels, and no human or LLM judgement of a pair is an input.

5. Limitations and "if you don't know"

The honest-limitations answers

The pattern is always the same: **say the limitation plainly first, then the mitigation, then stop.** These are disclosed in our Documentation (Section 5 and Appendix C), so saying them is a strength, not a risk. The full answers are in the linked questions.

#	Limitation (say it first)	Mitigation (then this)	Full answer
1	No untouched end-to-end validation of the final file. The held-out scores exclude the second-chance searches and the France rules, folds 8 and 9 also chose thresholds and the consistency check, and the final US / India models were refitted on folds 6-9.	Held-out per-business F0.5 from fits that leave the scored fold out: US 0.991212 / 0.991545, India 0.990046 / 0.990298, the same range as the public 0.9913.	Q181
2	France has no labels, so we have no measured French F0.5.	French readers self-trained with labelled US / Indian replay; a stricter 0.82 threshold; the France recipe on labelled US / Indian rows at 0.82 scores 0.990508 / 0.989467 on fold 9 (a proxy, not a French score).	Q158 (full guide)
3	The France rules were designed after analysing the unlabelled French test records, including AI-assisted review of sampled pairs, and some parameters were chosen after seeing test results, including public scores.	Each rule is code on model scores, raw text and the address; no test label and no human or AI judgement of any pair is an input; the evidence is label-free control groups; the largest action removes 29,021 of 887,630 accepted French links.	Q182
4	We kept the French E5 reader although it failed two of its four written checks (the generated French test set and the expected-gain rule).	It passed US / Indian quality (AUC 0.998544 vs 0.998570) and pseudo-label agreement (0.999206); it changes French links only (-2,228 / +3,396); it was a leaderboard-driven decision and we disclose it.	Q183
5	Public-leaderboard feedback influenced selections: 18 uploads, the final file chosen by public score, the refit kept and the France refit dropped on public scores.	US / India thresholds came from held-out folds; held-out scores sit in the same range; the late public gains were small (0.991114 → 0.991300).	Q153
6	The pipeline computes statistics on the unlabelled test files (word rarity, indexes, French geography, self-training, the French threshold from link volume), so our evaluation is transductive.	No test labels are ever used, every such use is listed in our Documentation, and in production the reference list and the incoming stream are available at run time anyway.	Q184 (full guide)
7	The unseen-country route is untested on real data.	It is conservative by design: the MiniLM reader, a decision model trained on labelled US / Indian rows, threshold 0.82; a new market would get its own cleaner, self-trained readers and a small labelled sample.	Q162
8	No clean ablation of the "what changed" features and no feature-importance ranking.	Closest evidence: generic inputs replacing data-specific ones moved the public score 0.990881 → 0.990934; we would run grouped permutation importance on held-out folds.	Q102
9	Single-record latency was not measured; our runs were batch jobs.	Work per record is bounded at every stage (on average 1.17 reader pairs per record); batch: about 1.3 h index + ranker and 1.4 h readers for 9.97 million records.	Q210
10	(Only if pressed.) The two second-chance models' training records were selected with earlier decision models whose inputs included data-specific features we later removed.	Disclosed in our Documentation; the fix would be to re-select those records with the final decision models and retrain.	Q188 (full guide)

One sentence that covers the first four, for when time is short: "We had no untouched end-to-end validation of the final file, France has no labels and its rules were designed after looking at unlabelled test records, and we kept the French E5 reader although it failed two of its four written checks; all of this is disclosed, and close versions of our models score in the same range on held-out folds."

If you don't know: how to answer gracefully

Senior scientists respect "we did not measure that" far more than a guessed number. A confident, honest non-answer followed by what you *would* measure is a good answer.

Phrases that work.

Situation	Say
The number was never measured	"We did not measure that, so I won't guess. What we would measure is ... on a held-out fold, with the exact per-business F0.5."
The number exists but you do not remember it	"I don't want to misquote it; it is in Section ... of our Documentation, and I'm happy to send it right after."
A good idea you did not try	"That's a good idea, and we didn't try it. We'd judge it the same way as everything else: same shortlist, held-out fold 9, per-business F0.5, and a paired bootstrap over businesses."
The question is unclear	"Just to make sure I answer the right question: do you mean ... or ...?"
It touches a weakness	"Honestly, that is a limitation: ... What limits the damage is ..."
You are asked for a comparison we lack (for example dense retrieval, BM25, XLM-R large, a ranking loss)	"We did not benchmark it, so I can't claim ours is better. Our reason for the choice was ..., and here is how we'd compare them."
Something was decided on public scores	"That one was a leaderboard-driven decision, and our Documentation discloses it."
A question on near-identical look-alike businesses or hard negatives	Use the wording of Q115 exactly, and add nothing.
Team roles	Use the one sentence agreed with Monika and Hemachand before 7 October (Q228).
AI assistance	Answer briefly and factually, as in Q229 .
You are running long	"The short answer is ...; I'm happy to go deeper if useful." Then stop.
You realise you misspoke	"Let me correct that: the exact number is ..." Correcting yourself is fine.

What never to say.

- Anything about near-identical look-alike businesses beyond the agreed wording of [Q115](#), and never raise the topic yourself.
- Internal code names or rule letters (see the code-name table in Part C of the full guide), or a record id such as "S1-...".
- A number that is not in the numbers table in Part C of the full guide or in a question's **Numbers** list; never quote toy or illustrative numbers.
- "About 130 per business in the final run" (it is from the blocking study of 25 September; the final run reports 21.7107 after the quick filter).
- "The ranker uses a ranking loss" (it is a pointwise classifier with log loss).
- "The 0.4 lead was never tuned" or "the 0.4 lead was tuned" (say: one fixed value for every country; the threshold was tuned on fold 8).
- "Our held-out scores are an end-to-end test" (they exclude the second-chance searches and the France rules).
- "The French readers are 99.9 % accurate" (0.999206 is agreement with their own teachers' pseudo-labels, not accuracy).
- Any break-even story for the consistency check's bars (say: the four bars, 0.75 / 0.68 / 0.72 / 0.77, were tuned on folds 8 and 9).
- "One LightGBM call per record" (say: a handful of small tree-model calls per pair, five seeds).

Three breaths before any hard question. (1) What is the judge really testing? (2) Which one number proves my answer? (3) Is there a limitation I should say first?

Three answers with agreed wording

The phrases above point to these three answers. Learn them as they are; Q228 still needs the sentence from the to-do list.

Q115. How do the decision models learn to reject near-identical look-alike businesses? ★★★

Why they ask: Look-alike businesses are a main source of wrong links, so they test your defence. Answer only if asked; never volunteer it.

SAY Mainly through the "what changed" features and the lead rule. In addition, for the US and India only, the decision models were trained with extra hard examples: copies of unlinked training records that sit right next to a candidate business, with small name edits. How many to add was set from label-free counts on the test records, never from labels, and our documentation discloses it; the France decision model uses no such extra examples.

IF THEY PUSH It is standard hard-example augmentation: more training examples of the confusable case near the decision boundary, where an ordinary sample has few. The safeguard is that the amount was set without any labels, so no test answer enters training.

IF THEY PUSH FURTHER Synthetic training pairs were allowed by the competition rules. Every held-out decision-model F0.5 we quote is computed on real labelled records only, never on the added examples, so the added rows can change what the model learns but cannot flatter that evaluation.

Q228. Who did what in your team? ★

Why they ask: They want to know the presenter's own role and that the team understands the work.

SAY We are a team of three, Monika, Hemachand and I, and we built and checked the pipeline together; I am presenting it today. The documentation, code and evidence files are our shared work, and all three of us stand behind them. **TO FILL IN** (*before 7 October: one accurate sentence, agreed with Monika and Hemachand, on the split of work. Our documents do not record roles, so do not improvise one on stage.*)

IF THEY PUSH We are happy to share a breakdown afterwards; the full method, code and disclosures are in our documentation and repository.

Q229. Did you use AI tools to build this? ★★

Why they ask: They want a straight, factual answer; our documentation already discloses it.

SAY Yes, and our documentation discloses it: AI coding assistants wrote and ran code and analysis scripts during development. For three later France rule changes, the cells of the cell veto, the acronym rescue and the lower bound of the generic-word rescue, they also chose the cells and thresholds and wrote the rule code, which we adopted. No LLM runs in the submitted pipeline.

IF THEY PUSH Their output went through the same checks as the rest of our work: held-out folds where labels exist, label-free tests and public scores for the France rules, written acceptance checks for new readers, and the organisers' validator. We can walk through any part of the code and the reason for each choice.

IF THEY PUSH FURTHER The three changes were the cell-veto cells, the acronym rescue and the lower bound of the generic-word rescue. Every rule is code on model scores and raw text; no human or AI judgement of a pair is an input.

6. Key numbers

The headline rows are from the numbers table of the full guide (Part C (a)); the rest is the "Numbers to have ready" table of the talk's prep sheet. Say numbers exactly as written.

The headline

Item	Number	Say	Source
Public F0.5 of the submitted file	0.991300, upload 16 of 18	"0.9913"	Doc 5, B.6
First scored baseline	0.968212 (upload 2)	"0.968"	Doc B.6
Held-out F0.5, fold 9 / fold 8 (close versions of our decision models: the India recipe with 3 of its 5 seeds; excludes second-chance searches and France rules)	US 0.991212 / 0.991545; India 0.990046 / 0.990298	"between 0.990 and 0.992 on hidden folds"	Doc 5
France recipe proxy (one fit, applied to US / Indian rows at 0.82, no consistency check; not a French score)	fold 9 0.990508 / 0.989467; fold 8 0.990977 / 0.989850	"a proxy, not a French score"	Doc 5
Measured French F0.5	none (France has no labels)	"we have no measured French F0.5"	Doc 2.1, 5

Numbers to have ready

Topic	Number	Source
Test size	1,732,544 businesses; 9,969,589 incoming records; France 259,452 / 1,434,993, no labels	Doc 2.1
Comparison space	6,724,569,566,212 same-country pairs; reduction ratio 0.99999817	Doc 3
Funnel per business	about 130 (index lookup, blocking study) → 21.71 (quick rule-based filter) → 6.75 (shortlist, derived) → 7.10 (candidate file, 12,293,019 pairs)	Doc 3
Per incoming record	1.23 candidates; 1.17 pairs reach the readers (derived); 90.59 % have exactly one candidate	Doc B.1
Recall, train fold 9	index 99.3694 % US / 98.5068 % India; shortlist 98.8748-99.3628 % / 98.0572-98.4831 %	Doc B.2
Cost of the quick filter	true business kept for 99.974 % of held-out records; fold-9 F0.5 0.990117 → 0.990086	Doc 3
Folds	10 folds per business (hash of the business id)	Doc 4
Training splits	ranker folds 0-5, early stopping on 6; readers folds 0-5 (1 epoch, bf16); decision models fit on 6-7, early stopping on 8, US/India refit on 6-9, mean of 5 seeds; native-script rescue folds 0-7; address rescue 6-7	Doc 4
Decision	best candidate only if score ≥ 0.75 (US, India; tuned on fold 8) or 0.82 (France, default) and lead over the runner-up ≥ 0.4 (one fixed value for every country); consistency check 0.75 / 0.68 / 0.72 / 0.77 for 0 / 1 / 2 / 3+ other accepted records (folds 8 and 9); rescues 0.88 native script, 0.86 US address (fold 8)	Doc 4
French E5 reader checks	US/India quality within tolerance: pass; agreement with held-out French pseudo-labels 0.999206: pass; generated labelled French test set 0.910516 vs 0.914964: fail; "expected gain safely positive" rule, pessimistic -0.0001806: fail	Doc B.4
Kept / dropped (public)	US/India refit 0.990621 → 0.990676 kept; France refit 0.990726 dropped; second French round 0.991277 dropped; final generic inputs (upload 14) 0.990881 → 0.990934	Doc B.6
Links	5,856,936 links; 100,087 empty rows	Doc B.1
Second-chance links	native script +9,759 (India); address +192 (US), +2,945 (India)	Doc B.1
France	887,630 accepted by the decision model → 867,559 links after the France rules	Doc B.3, B.1
Compute	full retrain about 11.5 h, 1× RTX 4070 Ti SUPER 16 GB, 32 GB RAM, about 70 GB disk; inference: blocking + ranker about 1.3 h, reader scoring (all except the French E5 reader) about 1.4 h	READMEs; clean-run report
Reproduction	from the trained models the inference stages rewrite both files byte for byte; a full retrain reproduced 99.82 % of the links, validator PASS	READMEs

7. Before the finale: to do

Three things to settle before you walk on stage. Tick them off on Tuesday evening at the latest.

☐ **Agree one accurate sentence on the team's split of work.** Agree it with Monika and Hemachand and write it into the answer to [Q228](#) ("Who did what in your team?"), which shows **TO FILL IN** where the sentence goes. Our documents do not record roles, so do not improvise one on stage.

☐ **Know what is "background" and what is Documentation.** Items marked "background" or "(code)", and facts whose source is Code, CodeREADME or Evidence, are true but are not in our submitted Documentation. Use them only if a judge asks directly; on any conflict the Documentation wins.

☐ **Never say a record id aloud.** The worked examples print ids such as "S1-..." (a business) or "S2-..." (an incoming record) only so that the records can be found in our files. On stage say "a US business", "an incoming record from Nantes" and so on.

Our sentence on the split of work (write it here, then say it as the last sentence of [Q228](#)):